

LeetMastery			Contents					
By Pattern								
Python 01 • Print Edition • 9-Link Landscape								
311 questions • 21 patterns • 140 black • 9 up-linkage								
Designations • Images • Row Names • All of them • Sorted Best-worst								

[illegible][illegible]

You are given an integer n and a helper function `knows` bool n tells you whether a knows b. Implement a function `findTeacher()`. There will be exactly one candidate if there is no tie.

Return the candidate's label if there is a candidate at the party. If there is no candidate, return `-1`.

Note that there is a $2 \leq n \leq 20$ array graph given as input is there always exists a solution and only one solution through the helper function `knows`.

`graph[i][j] == 1` represents person i knows person j , otherwise `graph[i][j] == 0`.

Example 1

```

graph TD
    0((0))
    1((1))
    2((2))
    1 --> 0
    2 --> 0
  
```

Input: `graph = [[1,0,0],[0,0,0],[0,0,0]]`
 Output: `0`

Notes: There are three persons labeled with 0, 1 and 2. `graph[0][1]` is a zero person 0 does not know person 1. `graph[0][2]` is a zero person 0 does not know person 2. The candidate is the person labeled as a known person 1 and 2 know him but, otherwise not know anyone.

Example 2

```

graph TD
    0((0))
    1((1))
    2((2))
    1 --> 0
    2 --> 1
  
```

Input: `graph = [[1,0,0],[0,1,0],[0,0,0]]`
 Output: `1`

Notes: There are three persons labeled with 0, 1 and 2. `graph[0][1]` is a zero person 0 does not know person 1. `graph[1][2]` is a zero person 1 does not know person 2. The candidate is the person labeled as a known person 1 and 2 know him but, otherwise not know anyone.

Input: `graph = [[1,0,0],[0,1,0],[0,0,0]]`
 Output: `1`

Notes: There are three persons labeled with 0, 1 and 2. `graph[0][1]` is a zero person 0 does not know person 1. `graph[1][2]` is a zero person 1 does not know person 2. The candidate is the person labeled as a known person 1 and 2 know him but, otherwise not know anyone.

Example 3

```

graph TD
    0((0))
    1((1))
    2((2))
    1 --> 0
    2 --> 0
    2 --> 1
  
```

Input: `graph = [[1,0,0],[0,1,0],[0,0,1]]`
 Output: `-1`

Notes: There are three persons labeled with 0, 1 and 2. `graph[0][1]` is a zero person 0 does not know person 1. `graph[0][2]` is a zero person 0 does not know person 2. `graph[1][2]` is a one person 1 knows person 2. The candidate is the person labeled as a known person 1 and 2 know him but, otherwise not know anyone.

Python

```

def findTeacher():
    # graph[0][0] == 1
    # graph[0][1] == 0
    # graph[0][2] == 0
    # graph[1][0] == 0
    # graph[1][1] == 0
    # graph[1][2] == 0
    # graph[2][0] == 0
    # graph[2][1] == 0
    # graph[2][2] == 0
    # graph[3][0] == 0
    # graph[3][1] == 0
    # graph[3][2] == 0
    # graph[4][0] == 0
    # graph[4][1] == 0
    # graph[4][2] == 0
    # graph[5][0] == 0
    # graph[5][1] == 0
    # graph[5][2] == 0
    # graph[6][0] == 0
    # graph[6][1] == 0
    # graph[6][2] == 0
    # graph[7][0] == 0
    # graph[7][1] == 0
    # graph[7][2] == 0
    # graph[8][0] == 0
    # graph[8][1] == 0
    # graph[8][2] == 0
    # graph[9][0] == 0
    # graph[9][1] == 0
    # graph[9][2] == 0
    # graph[10][0] == 0
    # graph[10][1] == 0
    # graph[10][2] == 0
    # graph[11][0] == 0
    # graph[11][1] == 0
    # graph[11][2] == 0
    # graph[12][0] == 0
    # graph[12][1] == 0
    # graph[12][2] == 0
    # graph[13][0] == 0
    # graph[13][1] == 0
    # graph[13][2] == 0
    # graph[14][0] == 0
    # graph[14][1] == 0
    # graph[14][2] == 0
    # graph[15][0] == 0
    # graph[15][1] == 0
    # graph[15][2] == 0
    # graph[16][0] == 0
    # graph[16][1] == 0
    # graph[16][2] == 0
    # graph[17][0] == 0
    # graph[17][1] == 0
    # graph[17][2] == 0
    # graph[18][0] == 0
    # graph[18][1] == 0
    # graph[18][2] == 0
    # graph[19][0] == 0
    # graph[19][1] == 0
    # graph[19][2] == 0
    # graph[20][0] == 0
    # graph[20][1] == 0
    # graph[20][2] == 0
    # graph[21][0] == 0
    # graph[21][1] == 0
    # graph[21][2] == 0
    # graph[22][0] == 0
    # graph[22][1] == 0
    # graph[22][2] == 0
    # graph[23][0] == 0
    # graph[23][1] == 0
    # graph[23][2] == 0
    # graph[24][0] == 0
    # graph[24][1] == 0
    # graph[24][2] == 0
    # graph[25][0] == 0
    # graph[25][1] == 0
    # graph[25][2] == 0
    # graph[26][0] == 0
    # graph[26][1] == 0
    # graph[26][2] == 0
    # graph[27][0] == 0
    # graph[27][1] == 0
    # graph[27][2] == 0
    # graph[28][0] == 0
    # graph[28][1] == 0
    # graph[28][2] == 0
    # graph[29][0] == 0
    # graph[29][1] == 0
    # graph[29][2] == 0
    # graph[30][0] == 0
    # graph[30][1] == 0
    # graph[30][2] == 0
    # graph[31][0] == 0
    # graph[31][1] == 0
    # graph[31][2] == 0
    # graph[32][0] == 0
    # graph[32][1] == 0
    # graph[32][2] == 0
    # graph[33][0] == 0
    # graph[33][1] == 0
    # graph[33][2] == 0
    # graph[34][0] == 0
    # graph[34][1] == 0
    # graph[34][2] == 0
    # graph[35][0] == 0
    # graph[35][1] == 0
    # graph[35][2] == 0
    # graph[36][0] == 0
    # graph[36][1] == 0
    # graph[36][2] == 0
    # graph[37][0] == 0
    # graph[37][1] == 0
    # graph[37][2] == 0
    # graph[38][0] == 0
    # graph[38][1] == 0
    # graph[38][2] == 0
    # graph[39][0] == 0
    # graph[39][1] == 0
    # graph[39][2] == 0
    # graph[40][0] == 0
    # graph[40][1] == 0
    # graph[40][2] == 0
    # graph[41][0] == 0
    # graph[41][1] == 0
    # graph[41][2] == 0
    # graph[42][0] == 0
    # graph[42][1] == 0
    # graph[42][2] == 0
    # graph[43][0] == 0
    # graph[43][1] == 0
    # graph[43][2] == 0
    # graph[44][0] == 0
    # graph[44][1] == 0
    # graph[44][2] == 0
    # graph[45][0] == 0
    # graph[45][1] == 0
    # graph[45][2] == 0
    # graph[46][0] == 0
    # graph[46][1] == 0
    # graph[46][2] == 0
    # graph[47][0] == 0
    # graph[47][1] == 0
    # graph[47][2] == 0
    # graph[48][0] == 0
    # graph[48][1] == 0
    # graph[48][2] == 0
    # graph[49][0] == 0
    # graph[49][1] == 0
    # graph[49][2] == 0
    # graph[50][0] == 0
    # graph[50][1] == 0
    # graph[50][2] == 0
    # graph[51][0] == 0
    # graph[51][1] == 0
    # graph[51][2] == 0
    # graph[52][0] == 0
    # graph[52][1] == 0
    # graph[52][2] == 0
    # graph[53][0] == 0
    # graph[53][1] == 0
    # graph[53][2] == 0
    # graph[54][0] == 0
    # graph[54][1] == 0
    # graph[54][2] == 0
    # graph[55][0] == 0
    # graph[55][1] == 0
    # graph[55][2] == 0
    # graph[56][0] == 0
    # graph[56][1] == 0
    # graph[56][2] == 0
    # graph[57][0] == 0
    # graph[57][1] == 0
    # graph[57][2] == 0
    # graph[58][0] == 0
    # graph[58][1] == 0
    # graph[58][2] == 0
    # graph[59][0] == 0
    # graph[59][1] == 0
    # graph[59][2] == 0
    # graph[60][0] == 0
    # graph[60][1] == 0
    # graph[60][2] == 0
    # graph[61][0] == 0
    # graph[61][1] == 0
    # graph[61][2] == 0
    # graph[62][0] == 0
    # graph[62][1] == 0
    # graph[62][2] == 0
    # graph[63][0] == 0
    # graph[63][1] == 0
    # graph[63][2] == 0
    # graph[64][0] == 0
    # graph[64][1] == 0
    # graph[64][2] == 0
    # graph[65][0] == 0
    # graph[65][1] == 0
    # graph[65][2] == 0
    # graph[66][0] == 0
    # graph[66][1] == 0
    # graph[66][2] == 0
    # graph[67][0] == 0
    # graph[67][1] == 0
    # graph[67][2] == 0
    # graph[68][0] == 0
    # graph[68][1] == 0
    # graph[68][2] == 0
    # graph[69][0] == 0
    # graph[69][1] == 0
    # graph[69][2] == 0
    # graph[70][0] == 0
    # graph[70][1] == 0
    # graph[70][2] == 0
    # graph[71][0] == 0
    # graph[71][1] == 0
    # graph[71][2] == 0
    # graph[72][0] == 0
    # graph[72][1] == 0
    # graph[72][2] == 0
    # graph[73][0] == 0
    # graph[73][1] == 0
    # graph[73][2] == 0
    # graph[74][0] == 0
    # graph[74][1] == 0
    # graph[74][2] == 0
    # graph[75][0] == 0
    # graph[75][1] == 0
    # graph[75][2] == 0
    # graph[76][0] == 0
    # graph[76][1] == 0
    # graph[76][2] == 0
    # graph[77][0] == 0
    # graph[77][1] == 0
    # graph[7
```

Return the minimum number of monsters needed to take all cities. If there is no way to take all the cities, return -1. Example 1: 	Input: n = 3, relations = [[1,2],[2,3]] Output: 2 Explanation: The figure above represents the given graph. In the figure, monster, you can take cities 1 and 2. If you take monster, you can take city 3. Example 2: 	Input: n = 3, relations = [[1,2],[2,3],[3,1]] Output: -1 Explanation: No course can be created because they are prerequisites of each other. Constraints: 1 ≤ n ≤ 5000 1 ≤ relations.length ≤ 5000 - relations[i].length == 2 0 ≤ i < relations.length, relations[i][0] < n - preCourse[i] = lastCourse - All the pairs [preCourse[i], lastCourse[i]] are unique. Source: Force (Python)
Graphs DFS <pre> # Back-Track Algorithm class Solution: def minCourses(self, relations, relationsEq): # Create Graph: For all nodes, its relationsEq graph = {} for i in range(1, n + 1): graph[i] = [] for i, j in relationsEq: graph[i].append(j) # Now we can graph traversal def dfs(course): visited.add(course) if not graph[course]: return 1 else: for i in graph[course]: if i not in visited: dfs(i) return 1 + dfs(course) return dfs(1) </pre>	Graphs DFS <pre> class Solution: def minCourses(self, relations): n = len(relations) graph = {} for i, j in relations: graph[i].append(j) def dfs(course): visited.add(course) if not graph[course]: return 1 else: for i in graph[course]: if i not in visited: dfs(i) return 1 + dfs(course) return dfs(1) </pre>	Graphs DFS <pre> # Back-Track Algorithm class Solution: def minCourses(self, relations): n = len(relations) graph = {} for i, j in relations: graph[i].append(j) def dfs(course): visited.add(course) if not graph[course]: return 1 else: for i in graph[course]: if i not in visited: dfs(i) return 1 + dfs(course) return dfs(1) </pre>
Commonality Solutions (Python) DFS <pre> class Solution: def minCourses(self, relations): n = len(relations) graph = {} for i, j in relations: graph[i].append(j) def dfs(course): visited.add(course) if not graph[course]: return 1 else: for i in graph[course]: if i not in visited: dfs(i) return 1 + dfs(course) return dfs(1) </pre>	Graphs DFS <pre> class Solution: def minCourses(self, relations): n = len(relations) graph = {} for i, j in relations: graph[i].append(j) def dfs(course): visited.add(course) if not graph[course]: return 1 else: for i in graph[course]: if i not in visited: dfs(i) return 1 + dfs(course) return dfs(1) </pre>	Graphs DFS <pre> class Solution: def minCourses(self, relations): n = len(relations) graph = {} for i, j in relations: graph[i].append(j) def dfs(course): visited.add(course) if not graph[course]: return 1 else: for i in graph[course]: if i not in visited: dfs(i) return 1 + dfs(course) return dfs(1) </pre>

<pre> for var, count in relations: m[var] += count m[reverse(var)] -= count # A bipartite graph with all vertices having an equal number of # incident edges is called a regular graph. def isRegularGraph(G): """Returns True if G is a regular graph, False otherwise""" n = G.number_of_vertices() m = G.degree() return len(set(m)) == 1 and n > 0 </pre>	<pre> # Incremental algorithm for graph coloring vertices. def greedy_coloring(G): """Returns a greedy coloring of the graph G""" m = {} for v in G.vertices(): m[v] = 0 for u in G.neighbors(v): if m[u] > 0: m[v] = max(m[u] + 1, m[v]) for v in G.vertices(): m[v] = 0 for u in G.neighbors(v): if m[u] > 0: m[v] = max(m[u] + 1, m[v]) return m </pre>	<pre> while True: for i in range(1, n): for j in range(i+1, n): if not isEdge(i, j): return i, j return None </pre>									
Graphs #305 Number of Islands II LeetCode - Simplest - WALKER - LeetCode Answer - Hash Table - Count - Matrix Logic - Process the 0s in the matrix and count the number of operations and it is the answer. Additions of 1s are not counted since they are not part of the island. Problem We are given an empty 2D binary grid <code>land</code> of size <code>m x n</code>. The grid represents a map where 0's represent water and 1's represent land. Initially, we are given an empty map (all water), and we need to calculate the number of islands. We may perform an add operation which turns the water at position <code>(x, y)</code> into a land. We are given an array of positions where <code>positions[i] = (x, y)</code> is the position <code>(i, j)</code> at which we should operate. Return an array of integers answer where <code>answer[i]</code> is the number of islands after turning the <code>(x[i], y[i])</code> into a land.	An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water. Example 1: <table border="1" data-bbox="1688 763 1799 828"> <tr><td>3</td><td>4</td><td>5</td></tr> <tr><td>3</td><td>2</td><td>6</td></tr> <tr><td>2</td><td>2</td><td>1</td></tr> </table>	3	4	5	3	2	6	2	2	1	Graphs Example 2: Input: <code>positions = [[1,1],[1,2],[2,1],[2,2],[3,1]]</code> Output: <code>[1,2,3,4,5]</code> Initially, the map is filled with water. - Operation 1: addLand(1,1) turns the water at grid[1][1] into a land. We have 1 island. - Operation 2: addLand(1,2) turns the water at grid[1][2] into a land. We still have 1 island. - Operation 3: addLand(2,1) turns the water at grid[2][1] into a land. We still have 1 island. - Operation 4: addLand(2,2) turns the water at grid[2][2] into a land. We still have 1 island. - Operation 5: addLand(3,1) turns the water at grid[3][1] into a land. We have 5 islands. Example 3: Input: <code>positions = [[1,1],[2,1],[3,1],[3,2],[4,1]]</code> Output: <code>[1,2,3,3,4]</code> <code>m = len(m), positionLength = len(positions)</code> <code>n = len(n) = len(m)</code> <code>positionCount = len(positions) - 2</code> <code>m < 0 < n</code>
3	4	5									
3	2	6									
2	2	1									
Graphs Follow up: Could you solve it in time complexity O(N log N), where <code>N = positions.length</code>? Brute Force (Python) # Write Three Algorithms 1. Brute Force 2. Brute Force 3. Brute Force 4. Brute Force 5. Brute Force 6. Brute Force 7. Brute Force 8. Brute Force 9. Brute Force 10. Brute Force 11. Brute Force 12. Brute Force 13. Brute Force 14. Brute Force 15. Brute Force 16. Brute Force 17. Brute Force 18. Brute Force 19. Brute Force 20. Brute Force 21. Brute Force 22. Brute Force 23. Brute Force 24. Brute Force 25. Brute Force 26. Brute Force 27. Brute Force 28. Brute Force 29. Brute Force 30. Brute Force 31. Brute Force 32. Brute Force 33. Brute Force 34. Brute Force 35. Brute Force 36. Brute Force 37. Brute Force 38. Brute Force 39. Brute Force 40. Brute Force 41. Brute Force 42. Brute Force 43. Brute Force 44. Brute Force 45. Brute Force 46. Brute Force 47. Brute Force 48. Brute Force 49. Brute Force 50. Brute Force 51. Brute Force 52. Brute Force 53. Brute Force 54. Brute Force 55. Brute Force 56. Brute Force 57. Brute Force 58. Brute Force 59. Brute Force 60. Brute Force 61. Brute Force 62. Brute Force 63. Brute Force 64. Brute Force 65. Brute Force 66. Brute Force 67. Brute Force 68. Brute Force 69. Brute Force 70. Brute Force 71. Brute Force 72. Brute Force 73. Brute Force 74. Brute Force 75. Brute Force 76. Brute Force 77. Brute Force 78. Brute Force 79. Brute Force 80. Brute Force 81. Brute Force 82. Brute Force 83. Brute Force 84. Brute Force 85. Brute Force 86. Brute Force 87. Brute Force 88. Brute Force 89. Brute Force 90. Brute Force 91. Brute Force 92. Brute Force 93. Brute Force 94. Brute Force 95. Brute Force 96. Brute Force 97. Brute Force 98. Brute Force 99. Brute Force 100. Brute Force	<pre> class Solution: def numIslands(self, grid: List[List[str]]) -> int: m = len(grid) n = len(grid[0]) visited = [[False] * n for _ in range(m)] count = 0 for i in range(m): for j in range(n): if not visited[i][j] and grid[i][j] == '1': count += 1 dfs(grid, i, j, visited) return count </pre>	<pre> def numIslands(grid: List[List[str]]) -> int: m = len(grid) n = len(grid[0]) visited = [[False] * n for _ in range(m)] count = 0 for i in range(m): for j in range(n): if not visited[i][j] and grid[i][j] == '1': count += 1 dfs(grid, i, j, visited) return count </pre>									
Community Solutions (Python) # WALKER	Graphs Short 8/23 - LeetCode by Python - Python Only - Blog - LeetCode	Graphs Short 8/23 - LeetCode by Python - Python Only - Blog - LeetCode									

[illegible]

<pre>def findShortest(s, t): n = len(s) m = len(t) dp = [[float('inf')] * (m + 1) for _ in range(n + 1)] dp[0][0] = 0 for i in range(1, n + 1): for j in range(1, m + 1): if s[i - 1] == t[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + 1 return dp[n][m]</pre> Time Complexity: $O(n \times m)$	<pre>def findShortest(s, t): n, m = len(s), len(t) dp = [[float('inf')] * (m + 1) for _ in range(n + 1)] dp[0][0] = 0 for i in range(1, n + 1): for j in range(1, m + 1): if s[i - 1] == t[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + 1 return dp[n][m]</pre> Time Complexity: $O(n \times m)$	<pre>def findShortest(s, t): n, m = len(s), len(t) dp = [[float('inf')] * (m + 1) for _ in range(n + 1)] dp[0][0] = 0 for i in range(1, n + 1): for j in range(1, m + 1): if s[i - 1] == t[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + 1 return dp[n][m]</pre> Time Complexity: $O(n \times m)$
Graphs 2153 String Transform Into Another String LeetCode - String - Medium Given two strings s and t, the length of the strings, because we can transform s into t by doing any of the following, how many steps do we need to transform s into t? Problem Given two strings s and t of the same length, determine whether we can transform s into t by doing any of the following operations. Constraints Return true if and only if we can transform s into t. Example 1:	<pre>def canTransform(s, t): n = len(s) for i in range(n): if s[i] != t[i]: return False return True</pre> Time Complexity: $O(n)$	<pre>def canTransform(s, t): n = len(s) for i in range(n): if s[i] != t[i]: return False return True</pre> Time Complexity: $O(n)$
Graphs	<pre>def canTransform(s, t): n = len(s) for i in range(n): if s[i] != t[i]: return False return True</pre> Time Complexity: $O(n)$	<pre>def canTransform(s, t): n = len(s) for i in range(n): if s[i] != t[i]: return False return True</pre> Time Complexity: $O(n)$
Graphs	<pre>def canTransform(s, t): n = len(s) for i in range(n): if s[i] != t[i]: return False return True</pre> Time Complexity: $O(n)$	<pre>def canTransform(s, t): n = len(s) for i in range(n): if s[i] != t[i]: return False return True</pre> Time Complexity: $O(n)$

4818 Go to file Return to file		» Quick Review - Graphs 5 questions - Insights - Complexity - Solutions 522 Longest Consecutive Sequence [Hard] Key Insights • Unlike an array, a hash table takes constant time look-ups. • Only count starting a sequence from a number that is the beginning of this previous number (i.e. it is not in set). • Each number is visited only once, ensuring an $O(n)$ time complexity. • Update the maximum sequence length as you discover longer consecutive sequences. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(n)$ We first iterate all the numbers into a hash set to enable $O(1)$ membership checks. For each number in the set, we check if it is the start of a sequence
Graphs	to ensure that the number minus one is not in the set, if it is the previous number, we skip it. Otherwise, we start a sequence by updating the maximum sequence length when necessary. This verified operation is the only operation necessary, ensuring the time complexity. 527 Find the Celebrity [Medium] Key Insights • A celebrity has no outgoing calls to other celebrities. • A celebrity knows everyone else, so celebrities cannot be the celebrity. • Verify the candidate by checking that they do not know anyone and that everyone knows them. • Verify the candidate in two passes: one for candidate selection and the other for validation. Space & Time Time Complexity: $O(n)$ - One pass for candidate selection and second pass for validation. Space Complexity: $O(1)$ - Only constant extra space is used.	Graphs
Key Insights • Use a greedy method (Dijkstra) to efficiently manage connected components (clusters). • Add all nodes and edges to a DS index. • Iterate over all nodes and check for unexplored links, done, left and perform union on all the links. • Avoid duplicate processing of the same cell to add multiple nodes. • Maintain a counter for the links that is updated during each operation. Space Complexity Time Complexity: $O(n^2)$ - n is the number of operations and n is the inverse Access Array. Space Complexity: $O(n)$ - n is the number of operations. Solution For creating the DS index 1. Add all nodes into the DS index that the structure is dynamically made known, where a new node is added. 2. Add all edges into the DS index that the structure is dynamically made known, where a new link is added and connected to the source. Check the four	Graphs	Graphs
Key Insights • Use a greedy method (Dijkstra) to efficiently manage connected components (clusters). • Add all nodes and edges to a DS index. • Iterate over all nodes and check for unexplored links, done, left and perform union on all the links. • Avoid duplicate processing of the same cell to add multiple nodes. • Maintain a counter for the links that is updated during each operation. Space Complexity Time Complexity: $O(n^2)$ - n is the number of operations and n is the inverse Access Array. Space Complexity: $O(n)$ - n is the number of operations. Solution For creating the DS index 1. Add all nodes into the DS index that the structure is dynamically made known, where a new node is added. 2. Add all edges into the DS index that the structure is dynamically made known, where a new link is added and connected to the source. Check the four	Graphs	Graphs
Graphs	Graphs	Graphs

#2 Greedy By Pattern - Python Only - 6 questions	#409 Longest Palindrome WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(1) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(1) Difficulty: Medium Problem Example 1 Example 2 Complexity	#279 Largest Number WACC - LeetCode Array - String - Greedy Time Complexity: O(N log N) Space Complexity: O(1) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - String - Greedy Time Complexity: O(N log N) Space Complexity: O(1) Difficulty: Medium Problem Example 1 Example 2 Complexity	#280 Wiggle Sort WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(1) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(1) Difficulty: Medium Problem Example 1 Example 2 Complexity	#234 Gas Station WACC - LeetCode Array - Greedy Time Complexity: O(N) Space Complexity: O(1) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - Greedy Time Complexity: O(N) Space Complexity: O(1) Difficulty: Medium Problem Example 1 Example 2 Complexity	#454 Array of Doubled Pairs WACC - LeetCode Array - Greedy Time Complexity: O(N log N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - Greedy Time Complexity: O(N log N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	#2131 Longest Palindrome by Concatenating Two Letter Words WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	#2626 Deobfuscate WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	#2638 Convert Object to JSON String WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	#2630 Memoize WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	#2675 Array of Objects to Matrix WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity	Community Solutions (Python) WACC - LeetCode Array - String - Greedy Time Complexity: O(N) Space Complexity: O(N) Difficulty: Medium Problem Example 1 Example 2 Complexity
--	--	---	--	---	---	---	--	--	---	--	---	---	--	---	--	---	--	---	---	---

[illegible][illegible]

p102 Reckoning Show ID: 105 License/Entry By Pattern Python Only Group ID: 8375 <pre> class Solution: def search(self, words: List[str], target: str) -> bool: # Sort the words in ascending order words.sort() # Binary search for the target left, right = 0, len(words) - 1 while left <= right: mid = (left + right) // 2 word = words[mid] if word == target: return True elif word < target: left = mid + 1 else: right = mid - 1 return False </pre> • Simplify	p103 Reckoning Show ID: 106 License/Entry By Pattern Python Only Group ID: 8376 <pre> def search(self, words: List[str], target: str) -> bool: # Sort the words in ascending order words.sort() # Binary search for the target left, right = 0, len(words) - 1 while left <= right: mid = (left + right) // 2 word = words[mid] if word == target: return True elif word < target: left = mid + 1 else: right = mid - 1 return False </pre> • Simplify	p104 Reckoning Show ID: 107 License/Entry By Pattern Python Only Group ID: 8377 <pre> def search(self, words: List[str], target: str) -> bool: # Sort the words in ascending order words.sort() # Binary search for the target left, right = 0, len(words) - 1 while left <= right: mid = (left + right) // 2 word = words[mid] if word == target: return True elif word < target: left = mid + 1 else: right = mid - 1 return False </pre> • Simplify	p105 Reckoning Show ID: 108 License/Entry By Pattern Python Only Group ID: 8378 <pre> def search(self, words: List[str], target: str) -> bool: # Sort the words in ascending order words.sort() # Binary search for the target left, right = 0, len(words) - 1 while left <= right: mid = (left + right) // 2 word = words[mid] if word == target: return True elif word < target: left = mid + 1 else: right = mid - 1 return False </pre> • Simplify	p106 Reckoning Show ID: 109 License/Entry By Pattern Python Only Group ID: 8379 <pre> def search(self, words: List[str], target: str) -> bool: # Sort the words in ascending order words.sort() # Binary search for the target left, right = 0, len(words) - 1 while left <= right: mid = (left + right) // 2 word = words[mid] if word == target: return True elif word < target: left = mid + 1 else: right = mid - 1 return False </pre> • Simplify
---	---	---	---	---

[illegible]

[illegible]

[illegible]

[illegible]

1 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84
--	---	---	---	---	--

[illegible]

Two Pointers #16 3Sum Closest NO LeetCode - Simplified - WACC - LeetCode Array - Two Pointers - Sorting Time Complexity: O(N^3) Space Complexity: O(1) or O(N) depending on the sorting algorithm (quicksort) to place array, so extra space might be needed Problem Given an integer array nums of length n and an integer target, find three integers in nums such that the sum of these three integers is closest to target. Return the sum of the three integers. You may assume that each input would have exactly one solution. Example 1: <pre>nums = [-1, 2, 1, -4] target = 1</pre> The sum of -1, 2, and -4 is -3, which is the closest to the target 1. (-1 + 2 + -4 = -3). Example 2: <pre>nums = [0, 0, 0] target = 0</pre> The sum of 0, 0, and 0 is 0, which is the closest to the target 0. (0 + 0 + 0 = 0). Example 3: <pre>nums = [1, 1, 1, 0] target = 1</pre> The sum of 1, 1, and 0 is 2, which is the closest to the target 1. (1 + 1 + 0 = 2). Example 4: <pre>nums = [1, 1, 1, 1] target = 4</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target 4. (1 + 1 + 1 = 3). Example 5: <pre>nums = [1, 1, 1, 1] target = 0</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target 0. (1 + 1 + 1 = 3). Example 6: <pre>nums = [1, 1, 1, 1] target = -1</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -1. (1 + 1 + 1 = 3). Example 7: <pre>nums = [1, 1, 1, 1] target = -2</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -2. (1 + 1 + 1 = 3). Example 8: <pre>nums = [1, 1, 1, 1] target = -3</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -3. (1 + 1 + 1 = 3). Example 9: <pre>nums = [1, 1, 1, 1] target = -4</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -4. (1 + 1 + 1 = 3). Example 10: <pre>nums = [1, 1, 1, 1] target = -5</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -5. (1 + 1 + 1 = 3). Example 11: <pre>nums = [1, 1, 1, 1] target = -6</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -6. (1 + 1 + 1 = 3). Example 12: <pre>nums = [1, 1, 1, 1] target = -7</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -7. (1 + 1 + 1 = 3). Example 13: <pre>nums = [1, 1, 1, 1] target = -8</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -8. (1 + 1 + 1 = 3). Example 14: <pre>nums = [1, 1, 1, 1] target = -9</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -9. (1 + 1 + 1 = 3). Example 15: <pre>nums = [1, 1, 1, 1] target = -10</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -10. (1 + 1 + 1 = 3). Example 16: <pre>nums = [1, 1, 1, 1] target = -11</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -11. (1 + 1 + 1 = 3). Example 17: <pre>nums = [1, 1, 1, 1] target = -12</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -12. (1 + 1 + 1 = 3). Example 18: <pre>nums = [1, 1, 1, 1] target = -13</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -13. (1 + 1 + 1 = 3). Example 19: <pre>nums = [1, 1, 1, 1] target = -14</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -14. (1 + 1 + 1 = 3). Example 20: <pre>nums = [1, 1, 1, 1] target = -15</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -15. (1 + 1 + 1 = 3). Example 21: <pre>nums = [1, 1, 1, 1] target = -16</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -16. (1 + 1 + 1 = 3). Example 22: <pre>nums = [1, 1, 1, 1] target = -17</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -17. (1 + 1 + 1 = 3). Example 23: <pre>nums = [1, 1, 1, 1] target = -18</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -18. (1 + 1 + 1 = 3). Example 24: <pre>nums = [1, 1, 1, 1] target = -19</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -19. (1 + 1 + 1 = 3). Example 25: <pre>nums = [1, 1, 1, 1] target = -20</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -20. (1 + 1 + 1 = 3). Example 26: <pre>nums = [1, 1, 1, 1] target = -21</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -21. (1 + 1 + 1 = 3). Example 27: <pre>nums = [1, 1, 1, 1] target = -22</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -22. (1 + 1 + 1 = 3). Example 28: <pre>nums = [1, 1, 1, 1] target = -23</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -23. (1 + 1 + 1 = 3). Example 29: <pre>nums = [1, 1, 1, 1] target = -24</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -24. (1 + 1 + 1 = 3). Example 30: <pre>nums = [1, 1, 1, 1] target = -25</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -25. (1 + 1 + 1 = 3). Example 31: <pre>nums = [1, 1, 1, 1] target = -26</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -26. (1 + 1 + 1 = 3). Example 32: <pre>nums = [1, 1, 1, 1] target = -27</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -27. (1 + 1 + 1 = 3). Example 33: <pre>nums = [1, 1, 1, 1] target = -28</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -28. (1 + 1 + 1 = 3). Example 34: <pre>nums = [1, 1, 1, 1] target = -29</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -29. (1 + 1 + 1 = 3). Example 35: <pre>nums = [1, 1, 1, 1] target = -30</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -30. (1 + 1 + 1 = 3). Example 36: <pre>nums = [1, 1, 1, 1] target = -31</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -31. (1 + 1 + 1 = 3). Example 37: <pre>nums = [1, 1, 1, 1] target = -32</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -32. (1 + 1 + 1 = 3). Example 38: <pre>nums = [1, 1, 1, 1] target = -33</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -33. (1 + 1 + 1 = 3). Example 39: <pre>nums = [1, 1, 1, 1] target = -34</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -34. (1 + 1 + 1 = 3). Example 40: <pre>nums = [1, 1, 1, 1] target = -35</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -35. (1 + 1 + 1 = 3). Example 41: <pre>nums = [1, 1, 1, 1] target = -36</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -36. (1 + 1 + 1 = 3). Example 42: <pre>nums = [1, 1, 1, 1] target = -37</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -37. (1 + 1 + 1 = 3). Example 43: <pre>nums = [1, 1, 1, 1] target = -38</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -38. (1 + 1 + 1 = 3). Example 44: <pre>nums = [1, 1, 1, 1] target = -39</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -39. (1 + 1 + 1 = 3). Example 45: <pre>nums = [1, 1, 1, 1] target = -40</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -40. (1 + 1 + 1 = 3). Example 46: <pre>nums = [1, 1, 1, 1] target = -41</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -41. (1 + 1 + 1 = 3). Example 47: <pre>nums = [1, 1, 1, 1] target = -42</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -42. (1 + 1 + 1 = 3). Example 48: <pre>nums = [1, 1, 1, 1] target = -43</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -43. (1 + 1 + 1 = 3). Example 49: <pre>nums = [1, 1, 1, 1] target = -44</pre> The sum of 1, 1, and 1 is 3, which is the closest to the target -44. (1 + 1 + 1 = 3). Example 50: <pre>nums = [1, 1, 1, 1] target = -45</pre> The sum of 1, 1, and 1 is 3, which is

#1000 Find Median from Data Streams Median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value and we take the lower of the two middle values. For example, for arr = [2,3,4,5,6], median is 4. For arr = [2,3,4,5], median is 3. Design a data structure that supports the following two operations: - addNum(int num): Add a new number num to the data structure. - findMedian(): Return the median of the current data structure. Example 1: <pre>addNum(1) addNum(2) findMedian() // returns 1.5 addNum(3) findMedian() // returns 2 addNum(4) findMedian() // returns 3 addNum(5) findMedian() // returns 3.5</pre> Constraints: 1 ≤ num ≤ 10⁵ - There will be at most 10⁵ calls to addNum and findMedian. Follow up: What if the number of calls to findMedian is 10⁵, will your solution still pass?	#1001 Sum of Subarray Ranges The range of an array is the difference between the largest and smallest element in the array. Return the sum of all subarray ranges. Example 1: <pre>Input: [1,2,3] Output: 4 Explanation: The subarray ranges are: [1]: 1 - 1 = 0 [2]: 2 - 2 = 0 [3]: 3 - 3 = 0 [1,2]: 2 - 1 = 1 [2,3]: 3 - 2 = 1 [1,2,3]: 3 - 1 = 2 Sum = 0 + 0 + 0 + 1 + 1 + 2 = 4</pre> Constraints: 1 ≤ nums.length ≤ 1000 0 ≤ nums[i] ≤ 1000	#1002 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1003 Check if Array is Sorted and Rotated Given an array nums, return true if the array is sorted in ascending order, or if it can be sorted by rotating it. Example 1: <pre>Input: [3,4,5,1,2] Output: true Explanation: The array [1,2,3,4,5] is sorted in ascending order.</pre> Constraints: 1 ≤ nums.length ≤ 100 0 ≤ nums[i] ≤ 100	#1004 Longest Common Prefix Write a function to find the longest common prefix string amongst an array of strings. Example 1: <pre>Input: ["flower","flow","flight"] Output: "fl"</pre> Constraints: 1 ≤ strs.length ≤ 100 1 ≤ strs[i].length ≤ 100 - strs[i] consists of lowercase English letters.	#1005 Maximize Profit from Stock Transactions Given an array prices where prices[i] is the price of a stock on day i, and an array fees where fees[i] is the fee of a transaction on day i. Return the maximum profit you can achieve. Example 1: <pre>Input: prices = [1,3,2,8,4,6], fees = [2,4,5,6,3,7] Output: 6 Explanation: Buy on day 1 (price 1) and sell on day 2 (price 3) for profit 2. Buy on day 4 (price 2) and sell on day 5 (price 8) for profit 6. Total profit = 2 + 6 = 8. After paying fees, the profit is 8 - 2 = 6.</pre> Constraints: 1 ≤ prices.length ≤ 1000 1 ≤ fees.length ≤ 1000 0 ≤ prices[i] ≤ 1000 0 ≤ fees[i] ≤ 1000	#1006 Check if Array is Sorted and Rotated Given an array nums, return true if the array is sorted in ascending order, or if it can be sorted by rotating it. Example 1: <pre>Input: [3,4,5,1,2] Output: true Explanation: The array [1,2,3,4,5] is sorted in ascending order.</pre> Constraints: 1 ≤ nums.length ≤ 100 0 ≤ nums[i] ≤ 100	#1007 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1008 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1009 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1010 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1011 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1012 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1013 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1014 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1015 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1016 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1017 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root = [4,9,0,5,1,8], target = 5, nodes = [4,9,8] Output: 8 Explanation: The node 8 is the closest to the target node 5.</pre> Constraints: 1 ≤ nodes.length ≤ 1000 1 ≤ nodes[i] ≤ 1000 1 ≤ target ≤ 1000	#1018 Find Closest Leaf to a Target in a Binary Tree Given a binary tree with root node 'root', a target node 'target', and an array of nodes 'nodes'. Return the value of the node that is closest to 'target' among the nodes in 'nodes'. Example 1: <pre>Input: root =</pre>
---	--	--	--	--	---	--	--	--	--	--	--	--	--	--	--	--	--	--

[illegible]

[illegible]

The diagram illustrates the mapping of a C program to a Haskell program. It consists of two columns of code, with arrows indicating the correspondence between them.

Left Column (C Program):

```

isPrime (Integer) -> Bool
isPrime = \n -> let sqrtN = sqrt (fromIntegral n) in
  do {
    let divisors = [2..sqrtN];
    let isDivisible = \d -> n `mod` d == 0;
    let isPrime = not (any isDivisible divisors);
    return isPrime
  }

```

Right Column (Haskell Program):

```

isPrime :: Integer -> Bool
isPrime = \n -> do {
  let sqrtN = sqrt (fromIntegral n);
  let divisors = [2..sqrtN];
  let isDivisible = \d -> n `mod` d == 0;
  let isPrime = not (any isDivisible divisors);
  return isPrime
}

```

Mapping Arrows:

- From C's `Integer` to Haskell's `Integer`.
- From C's `Bool` to Haskell's `Bool`.
- From C's `sqrt (fromIntegral n)` to Haskell's `sqrt (fromIntegral n)`.
- From C's `[2..sqrtN]` to Haskell's `[2..sqrtN]`.
- From C's `n `mod` d == 0` to Haskell's `n `mod` d == 0`.
- From C's `any isDivisible divisors` to Haskell's `any isDivisible divisors`.
- From C's `not (any isDivisible divisors)` to Haskell's `not (any isDivisible divisors)`.
- From C's `return isPrime` to Haskell's `return isPrime`.

[illegible][illegible][illegible][illegible][illegible][illegible]

Linked List

#343 Linked List Cycle

LeetCode - Easy

Medium

Linked List - Two Pointers

Time Complexity: $O(n)$ where n is the number of nodes in the linked list.

Space Complexity: $O(1)$ as we only maintain one node.

Problem

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Otherwise, there is no cycle in the list.

Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:

Example 2:

Input: head = [3,2,0,-4], pos = 1

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the first node (index=0).

Example 3:

Input: head = [1,2], pos = 0

Output: false

Explanation: There is a cycle in the linked list, where the tail connects to the first node (index=0).

Linked List

#141 Linked List Cycle II

LeetCode - Easy

Medium

Linked List - Two Pointers

Time Complexity: $O(n)$ where n is the number of nodes in the linked list.

Space Complexity: $O(1)$ as we only maintain one node.

Problem

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Otherwise, there is no cycle in the list.

Return the node where the cycle begins. If there is no cycle, return null.

Example 1:

QUESTION	ANSWER	QUESTION	ANSWER
1 <pre>graph LR; n(()) --- 1[1]; style n fill:none,stroke:#ccc,stroke-width:2px;</pre> Input: head = [1], pos = -4 Return Value: Explanation: There is no cycle in the linked list. Constraints: - The number of nodes in the list is in the range [0, 10⁴]. -100 ≤ Node.val ≤ 100 Can you solve it using O(1) space complexity? Continuously Submitter (Python) → NO	<pre><<= head if not head: return None node = head while node.next: node = node.next return node</pre> return False	<pre># Definition for singly-linked list. # class ListNode: # def __init__(self, x): # self.val = x # self.next = None class Solution: def hasCycle(self, head: Optional[ListNode]) -> bool: if not head: return False slow = head fast = head.next while slow != fast: slow = slow.next fast = fast.next if not fast: return False return True</pre>	

```

1 // Problem: Count number of submatrices with sum equal to target
2 // Input: Matrix of size N x M, Target sum
3 // Output: Number of submatrices with sum equal to target
4 // Example:
5 // Input: N = 4, M = 4, Target = 0
6 // Matrix:
7 // 1 0 1 0
8 // 0 1 1 1
9 // 0 1 0 1
10 // 1 1 0 0
11 // Output: 16
12 // Explanation: There are 16 submatrices with sum equal to 0.
13 // 1. [1, 0]
14 // 2. [0, 1]
15 // 3. [1, 1]
16 // 4. [0, 0]
17 // 5. [1, 0, 1, 0]
18 // 6. [0, 1, 1, 1]
19 // 7. [0, 1, 0, 1]
20 // 8. [1, 1, 0, 0]
21 // 9. [1, 0, 1, 0, 1, 0]
22 // 10. [0, 1, 1, 1, 0, 0]
23 // 11. [0, 1, 0, 1, 1, 1]
24 // 12. [1, 1, 0, 0, 0, 0]
25 // 13. [1, 0, 1, 0, 1, 0, 1, 0]
26 // 14. [0, 1, 1, 1, 0, 0, 1, 1]
27 // 15. [0, 1, 0, 1, 1, 1, 0, 0]
28 // 16. [1, 1, 0, 0, 0, 0, 1, 1]
29 // 17. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
30 // 18. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0]
31 // 19. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1]
32 // 20. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0]
33 // 21. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
34 // 22. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
35 // 23. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0]
36 // 24. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1]
37 // 25. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
38 // 26. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
39 // 27. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
40 // 28. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
41 // 29. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
42 // 30. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
43 // 31. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
44 // 32. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
45 // 33. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
46 // 34. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
47 // 35. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
48 // 36. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
49 // 37. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
50 // 38. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
51 // 39. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
52 // 40. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
53 // 41. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
54 // 42. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
55 // 43. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
56 // 44. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
57 // 45. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
58 // 46. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
59 // 47. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
60 // 48. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
61 // 49. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
62 // 50. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
63 // 51. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
64 // 52. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
65 // 53. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
66 // 54. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
67 // 55. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
68 // 56. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
69 // 57. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
70 // 58. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
71 // 59. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
72 // 60. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
73 // 61. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
74 // 62. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
75 // 63. [0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
76 // 64. [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0]
77 // 65. [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
78 // 66. [0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 
```

[illegible]

p2.75a	Matrix	p2.75b	Matrix	p2.75c
<pre>getColumns(cols) == 3 return ans # Python solution for counting number of submatrices that sum to target def numSubmat(mat, target): # Create a prefix sum array for each row for i in range(len(mat)): for j in range(1, len(mat[0])): mat[i][j] += mat[i][j-1] # Count the number of submatrices that sum to target ans = 0 for i in range(len(mat)): for j in range(1, len(mat[0])): for k in range(i+1, len(mat)): for l in range(j+1, len(mat[0])): if mat[i][j] + mat[k][l] == target: ans += 1 return ans</pre>	<pre>class Solution: def numSubmat(self, mat: List[List[int]]) -> int: # Create a prefix sum array for each row for i in range(len(mat)): for j in range(1, len(mat[0])): mat[i][j] += mat[i][j-1] # Count the number of submatrices that sum to target ans = 0 for i in range(len(mat)): for j in range(1, len(mat[0])): for k in range(i+1, len(mat)): for l in range(j+1, len(mat[0])): if mat[i][j] + mat[k][l] == target: ans += 1 return ans</pre>	<pre># Initialize a list to store the column sums between row 'top' and row 'bottom' col_sums = [] # Iterate over each row in the matrix for top in range(len(mat)): # Create a list to store the column sums for the current row row_sums = [] for bottom in range(top, len(mat)): # Calculate the column sums for the current row range for col in range(len(mat[0])): row_sums.append(mat[bottom][col]) # Append the row_sums to col_sums col_sums.append(row_sums) # Iterate over each column sum in col_sums for col_sum in col_sums: # Find the number of submatrices that sum to target count = 0 for i in range(len(col_sum)): for j in range(i+1, len(col_sum)): if col_sum[i] + col_sum[j] == target: count += 1 return count</pre>		

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

Q#173	Matrix	Q#174	Matrix	Q#175
Access Time Time Complexity: $O(N)$ Space Complexity: $O(1)$ The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited. Access Time Time Complexity: $O(N)$ Space Complexity: $O(1)$ The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited.	A simple double loop is sufficient to swap the indices, ensuring $O(N^2)$ time complexity. Time Complexity: $O(N^2)$ Space Complexity: $O(1)$ The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited. Access Time Time Complexity: $O(N)$ Space Complexity: $O(1)$ The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited.	The size of the output matrix is $(N - 2) \times (N - 2)$. For each valid cell (i, j) in the output matrix, the value is the 3 block in grid centered at $(i + 1, j + 1)$ in the input matrix. Time Complexity: $O(N^2)$ Space Complexity: $O(1)$ The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited.	Access Time Time Complexity: $O(N)$ Space Complexity: $O(1)$ The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited.	
173. Temperature Matrix [Easy] Difficulty: Easy The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited.	A simple double loop is sufficient to swap the indices, ensuring $O(N^2)$ time complexity. Time Complexity: $O(N^2)$ Space Complexity: $O(1)$ The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited.	The size of the output matrix is $(N - 2) \times (N - 2)$. For each valid cell (i, j) in the output matrix, the value is the 3 block in grid centered at $(i + 1, j + 1)$ in the input matrix. Time Complexity: $O(N^2)$ Space Complexity: $O(1)$ The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited.	Access Time Time Complexity: $O(N)$ Space Complexity: $O(1)$ The problem is solved by iterating through the matrix starting from the top-left corner and moving in a zig-zag pattern. The direction of movement is determined by the current row and column indices. The algorithm continues until all elements in the matrix have been visited.	

[illegible]

Q1757	Matrix	Q1758	Linked List	Q1759
	<pre>Input: links = [[1, 2], [2, 3], [3, 4], [4, 5]] Output: 5</pre> • The number of nodes in both lists is in the range [0, 50]. • 1000 or less nodes in total. • Both lists and links are stored in non-decreasing order. Community Solution (Python)	<pre>def longestCommonSubsequence(self, text1: str, text2: str) -> int: n, m = len(text1), len(text2) dp = [[0] * (m + 1) for _ in range(n + 1)] for i in range(1, n + 1): for j in range(1, m + 1): if text1[i - 1] == text2[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) return dp[n][m]</pre>	<pre>class Solution: def longestCommonSubsequence(self, text1: str, text2: str) -> int: n, m = len(text1), len(text2) dp = [[0] * (m + 1) for _ in range(n + 1)] for i in range(1, n + 1): for j in range(1, m + 1): if text1[i - 1] == text2[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) return dp[n][m]</pre>	<pre>class Solution: def longestCommonSubsequence(self, text1: str, text2: str) -> int: n, m = len(text1), len(text2) dp = [[0] * (m + 1) for _ in range(n + 1)] for i in range(1, n + 1): for j in range(1, m + 1): if text1[i - 1] == text2[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) return dp[n][m]</pre>

pa 750	Unlinked	pa 751	Unlinked	pa 752
<pre> Definition for singly-linked list. struct ListNode { int val; ListNode *next; ListNode() : val(0), next(nullptr) {} ListNode(int x) : val(x), next(nullptr) {} ListNode(int x, ListNode *next) : val(x), next(next) {} }; </pre>	<pre> current->next = 0; // Move the pointer to the next list // If there was a list to continue, attach the existing nodes from the other list if (list2) { current->next = list2; current->next->next = 0; } // The original list is used as its empty state return dummy.next; } }; </pre>	<pre> // Definition for singly-linked list. // class ListNode { // int val; // ListNode *next; // ListNode() {} // ListNode(int val) { this->val = val; } // ListNode(int val, ListNode *next) { this->val = val; this->next = next; } // }; </pre>	<pre> while (list1 list2) { // If list1 is null, then just take the other list if (!list1) return dummy->next->val; // If list2 is null, then just take the other list if (!list2) return list1->val; // If both are not null, compare the values if (list1->val < list2->val) { dummy->next->val = list1->val; list1 = list1->next; } else { dummy->next->val = list2->val; list2 = list2->next; } dummy->next = dummy->next->next; } return dummy->next; </pre>	<pre> // Definition for singly-linked list. // class ListNode { // int val; // ListNode *next; // ListNode() {} // ListNode(int val) { this->val = val; } // ListNode(int val, ListNode *next) { this->val = val; this->next = next; } // }; </pre>

[illegible][illegible]

name	g3777	linked list	g3778	linked list	g3779
<pre>class <name> { public: <name>() {} <name>(<name> const& n) {} ~<name>() {} <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; };</pre>	<pre> <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; <name>*<name>::next; <name>*<name>::head; <name>*<name>::tail; </pre>				

[illegible]

[illegible]

#130 Island Perimeter LeetCode - Simplify - WACC - LeetCode Along the perimeter of a 2D grid, there are some land cells (1) and some water cells (0). The perimeter of the island is the number of edges of the island that are adjacent to water cells (0). For example, the perimeter of the island in the image above is 16. Given a 2D grid map of 0s and 1s, return the perimeter of the island. Example 1: <pre>Input: grid = [[1,0,1,0,0,1], [0,0,0,0,0,0], [1,0,1,0,1,1], [0,1,1,0,0,0], [0,0,0,0,0,0]] Output: 16</pre> Example 2: <pre>Input: grid = [[0,0,0,0], [0,0,0,0], [0,0,0,0], [0,0,0,0]] Output: 0</pre> Constraints: <code>grid</code> is a 2D array of 0s and 1s. <code>1 <= grid.length <= 50</code> <code>1 <= grid[0].length <= 50</code>	#131 Path with Maximum Gold LeetCode - Simplify - WACC - LeetCode A gold mine is represented by a grid of size <code>m x n</code>. Each cell in the grid contains a non-negative integer representing the amount of gold in that cell. A path is defined as a sequence of cells starting from any cell and moving to adjacent cells (up, down, left, right) such that the path does not visit the same cell more than once. The goal is to find the maximum amount of gold that can be collected along any path. Example 1: <pre>Input: grid = [[0,6,0], [5,4,1], [0,3,0]] Output: 24</pre> Example 2: <pre>Input: grid = [[1,0,7], [2,0,6], [3,0,5], [4,0,4], [5,0,3], [6,0,2], [7,0,1]] Output: 28</pre> Constraints: <code>1 <= grid.length <= 15</code> <code>1 <= grid[0].length <= 15</code> <code>0 <= grid[i][j] <= 100</code>	#132 Shortest Path in a Grid with Obstacles LeetCode - Simplify - WACC - LeetCode A grid of size <code>m x n</code> is given, where each cell is either an empty space (0), an obstacle (1), or the starting point (2). The goal is to find the shortest path from the starting point to the bottom-right cell (m-1, n-1). The path can only move in the four cardinal directions (up, down, left, right) and cannot pass through obstacles. Example 1: <pre>Input: grid = [[0,1,0,0,0], [0,1,0,1,0], [0,0,0,1,0], [0,1,1,1,0], [0,0,0,0,0]] Output: 10</pre> Example 2: <pre>Input: grid = [[0,0,0], [0,0,0], [0,0,0]] Output: 4</pre> Constraints: <code>1 <= grid.length <= 20</code> <code>1 <= grid[0].length <= 20</code> <code>0 <= grid[i][j] <= 1</code> <code>grid</code> contains exactly one starting point (2).	#133 Clone Graph LeetCode - Simplify - WACC - LeetCode Given a reference of a node in a connected undirected graph, return a deep copy (clone) of the graph. Each node in the graph contains a value (int) and a list of its neighbors (List[Node]). Example 1: <pre>Input: node1 = 1 Output: 1</pre> Example 2: <pre>Input: node1 = 2 Output: 2</pre> Example 3: <pre>Input: node1 = 3 Output: 3</pre> Example 4: <pre>Input: node1 = 4 Output: 4</pre> Example 5: <pre>Input: node1 = 5 Output: 5</pre> Example 6: <pre>Input: node1 = 6 Output: 6</pre> Example 7: <pre>Input: node1 = 7 Output: 7</pre> Example 8: <pre>Input: node1 = 8 Output: 8</pre> Example 9: <pre>Input: node1 = 9 Output: 9</pre> Example 10: <pre>Input: node1 = 10 Output: 10</pre> Example 11: <pre>Input: node1 = 11 Output: 11</pre> Example 12: <pre>Input: node1 = 12 Output: 12</pre> Example 13: <pre>Input: node1 = 13 Output: 13</pre> Example 14: <pre>Input: node1 = 14 Output: 14</pre> Example 15: <pre>Input: node1 = 15 Output: 15</pre> Example 16: <pre>Input: node1 = 16 Output: 16</pre> Example 17: <pre>Input: node1 = 17 Output: 17</pre> Example 18: <pre>Input: node1 = 18 Output: 18</pre> Example 19: <pre>Input: node1 = 19 Output: 19</pre> Example 20: <pre>Input: node1 = 20 Output: 20</pre> Example 21: <pre>Input: node1 = 21 Output: 21</pre> Example 22: <pre>Input: node1 = 22 Output: 22</pre> Example 23: <pre>Input: node1 = 23 Output: 23</pre> Example 24: <pre>Input: node1 = 24 Output: 24</pre> Example 25: <pre>Input: node1 = 25 Output: 25</pre> Example 26: <pre>Input: node1 = 26 Output: 26</pre> Example 27: <pre>Input: node1 = 27 Output: 27</pre> Example 28: <pre>Input: node1 = 28 Output: 28</pre> Example 29: <pre>Input: node1 = 29 Output: 29</pre> Example 30: <pre>Input: node1 = 30 Output: 30</pre> Example 31: <pre>Input: node1 = 31 Output: 31</pre> Example 32: <pre>Input: node1 = 32 Output: 32</pre> Example 33: <pre>Input: node1 = 33 Output: 33</pre> Example 34: <pre>Input: node1 = 34 Output: 34</pre> Example 35: <pre>Input: node1 = 35 Output: 35</pre> Example 36: <pre>Input: node1 = 36 Output: 36</pre> Example 37: <pre>Input: node1 = 37 Output: 37</pre> Example 38: <pre>Input: node1 = 38 Output: 38</pre> Example 39: <pre>Input: node1 = 39 Output: 39</pre> Example 40: <pre>Input: node1 = 40 Output: 40</pre> Example 41: <pre>Input: node1 = 41 Output: 41</pre> Example 42: <pre>Input: node1 = 42 Output: 42</pre> Example 43: <pre>Input: node1 = 43 Output: 43</pre> Example 44: <pre>Input: node1 = 44 Output: 44</pre> Example 45: <pre>Input: node1 = 45 Output: 45</pre> Example 46: <pre>Input: node1 = 46 Output: 46</pre> Example 47: <pre>Input: node1 = 47 Output: 47</pre> Example 48: <pre>Input: node1 = 48 Output: 48</pre> Example 49: <pre>Input: node1 = 49 Output: 49</pre> Example 50: <pre>Input: node1 = 50 Output: 50</pre> Example 51: <pre>Input: node1 = 51 Output: 51</pre> Example 52: <pre>Input: node1 = 52 Output: 52</pre> Example 53: <pre>Input: node1 = 53 Output: 53</pre> Example 54: <pre>Input: node1 = 54 Output: 54</pre> Example 55: <pre>Input: node1 = 55 Output: 55</pre> Example 56: <pre>Input: node1 = 56 Output: 56</pre> Example 57: <pre>Input: node1 = 57 Output: 57</pre> Example 58: <pre>Input: node1 = 58 Output: 58</pre> Example 59: <pre>Input: node1 = 59 Output: 59</pre> Example 60: <pre>Input: node1 = 60 Output: 60</pre> Example 61: <pre>Input: node1 = 61 Output: 61</pre> Example 62: <pre>Input: node1 = 62 Output: 62</pre> Example 63: <pre>Input: node1 = 63 Output: 63</pre> Example 64: <pre>Input: node1 = 64 Output: 64</pre> Example 65: <pre>Input: node1 = 65 Output: 65</pre> Example 66: <pre>Input: node1 = 66 Output: 66</pre> Example 67: <pre>Input: node1 = 67 Output: 67</pre> Example 68: <pre>Input: node1 = 68 Output: 68</pre> Example 69: <pre>Input: node1 = 69 Output: 69</pre> Example 70: <pre>Input: node1 = 70 Output: 70</pre> Example 71: <pre>Input: node1 = 71 Output: 71</pre> Example 72: <pre>Input: node1 = 72 Output: 72</pre> Example 73: <pre>Input: node1 = 73 Output: 73</pre> Example 74: <pre>Input: node1 = 74 Output: 74</pre> Example 75: <pre>Input: node1 = 75 Output: 75</pre> Example 76: <pre>Input: node1 = 76 Output: 76</pre> Example 77: <pre>Input: node1 = 77 Output: 77</pre> Example 78: <pre>Input: node1 = 78 Output: 78</pre> Example 79: <pre>Input: node1 = 79 Output: 79</pre> Example 80: <pre>Input: node1 = 80 Output: 80</pre> Example 81: <pre>Input: node1 = 81 Output: 81</pre> Example 82: <pre>Input: node1 = 82 Output: 82</pre> Example 83: <pre>Input: node1 = 83 Output: 83</pre> Example 84: <pre>Input: node1 = 84 Output: 84</pre> Example 85: <pre>Input: node1 = 85 Output: 85</pre> Example 86: <pre>Input: node1 = 86 Output: 86</pre> Example 87: <pre>Input: node1 = 87 Output: 87</pre> Example 88: <pre>Input: node1 = 88 Output: 88</pre> Example 89: <pre>Input: node1 = 89 Output: 89</pre> Example 90: <pre>Input: node1 = 90 Output: 90</pre> Example 91: <pre>Input: node1 = 91 Output: 91</pre> Example 92: <pre>Input: node1 = 92 Output: 92</pre> Example 93: <pre>Input: node1 = 93 Output: 93</pre> Example 94: <pre>Input: node1 = 94 Output: 94</pre> Example 95: <pre>Input: node1 = 95 Output: 95</pre> Example 96: <pre>Input: node1 = 96 Output: 96</pre> Example 97: <pre>Input: node1 = 97 Output: 97</pre> Example 98: <pre>Input: node1 = 98 Output: 98</pre> Example 99: <pre>Input: node1 = 99 Output: 99</pre> Example 100: <pre>Input: node1 = 100 Output: 100</pre>
---	--	---	--

[illegible]

For neighbor in graph.neighbors(): if abs(x - neighbor.x) > 1 or abs(y - neighbor.y) > 1: continue # neighbor is adjacent # If neighbor is already visited, a cycle is detected if neighbor.is_visited(): return False # A neighbor might (if cycle is found in recursion, propagate value) if not return_value: return False # If not return_value is None, the graph is connected if return_value is None: return True # If all nodes are visited (visited == 1), the graph is connected return True # Example usage:	<pre>graph = Graph([(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 7), (7, 8), (8, 9), (9, 10), (10, 11), (11, 12), (12, 13), (13, 14), (14, 15), (15, 16), (16, 17), (17, 18), (18, 19), (19, 20), (20, 21), (21, 22), (22, 23), (23, 24), (24, 25), (25, 26), (26, 27), (27, 28), (28, 29), (29, 30), (30, 31), (31, 32), (32, 33), (33, 34), (34, 35), (35, 36), (36, 37), (37, 38), (38, 39), (39, 40), (40, 41), (41, 42), (42, 43), (43, 44), (44, 45), (45, 46), (46, 47), (47, 48), (48, 49), (49, 50), (50, 51), (51, 52), (52, 53), (53, 54), (54, 55), (55, 56), (56, 57), (57, 58), (58, 59), (59, 60), (60, 61), (61, 62), (62, 63), (63, 64), (64, 65), (65, 66), (66, 67), (67, 68), (68, 69), (69, 70), (70, 71), (71, 72), (72, 73), (73, 74), (74, 75), (75, 76), (76, 77), (77, 78), (78, 79), (79, 80), (80, 81), (81, 82), (82, 83), (83, 84), (84, 85), (85, 86), (86, 87), (87, 88), (88, 89), (89, 90), (90, 91), (91, 92), (92, 93), (93, 94), (94, 95), (95, 96), (96, 97), (97, 98), (98, 99), (99, 100), (100, 101), (101, 102), (102, 103), (103, 104), (104, 105), (105, 106), (106, 107), (107, 108), (108, 109), (109, 110), (110, 111), (111, 112), (112, 113), (113, 114), (114, 115), (115, 116), (116, 117), (117, 118), (118, 119), (119, 120), (120, 121), (121, 122), (122, 123), (123, 124), (124, 125), (125, 126), (126, 127), (127, 128), (128, 129), (129, 130), (130, 131), (131, 132), (132, 133), (133, 134), (134, 135), (135, 136), (136, 137), (137, 138), (138, 139), (139, 140), (140, 141), (141, 142), (142, 143), (143, 144), (144, 145), (145, 146), (146, 147), (147, 148), (148, 149), (149, 150), (150, 151), (151, 152), (152, 153), (153, 154), (154, 155), (155, 156), (156, 157), (157, 158), (158, 159), (159, 160), (160, 161), (161, 162), (162, 163), (163, 164), (164, 165), (165, 166), (166, 167), (167, 168), (168, 169), (169, 170), (170, 171), (171, 172), (172, 173), (173, 174), (174, 175), (175, 176), (176, 177), (177, 178), (178, 179), (179, 180), (180, 181), (181, 182), (182, 183), (183, 184), (184, 185), (185, 186), (186, 187), (187, 188), (188, 189), (189, 190), (190, 191), (191, 192), (192, 193), (193, 194), (194, 195), (195, 196), (196, 197), (197, 198), (198, 199), (199, 200), (200, 201), (201, 202), (202, 203), (203, 204), (204, 205), (205, 206), (206, 207), (207, 208), (208, 209), (209, 210), (210, 211), (211, 212), (212, 213), (213, 214), (214, 215), (215, 216), (216, 217), (217, 218), (218, 219), (219, 220), (220, 221), (221, 222), (222, 223), (223, 224), (224, 225), (225, 226), (226, 227), (227, 228), (228, 229), (229, 230), (230, 231), (231, 232), (232, 233), (233, 234), (234, 235), (235, 236), (236, 237), (237, 238), (238, 239), (239, 240), (240, 241), (241, 242), (242, 243), (243, 244), (244, 245), (245, 246), (246, 247), (247, 248), (248, 249), (249, 250), (250, 251), (251, 252), (252, 253), (253, 254), (254, 255), (255, 256), (256, 257), (257, 258), (258, 259), (259, 260), (260, 261), (261, 262), (262, 263), (263, 264), (264, 265), (265, 266), (266, 267), (267, 268), (268, 269), (269, 270), (270, 271), (271, 272), (272, 273), (273, 274), (274, 275), (275, 276), (276, 277), (277, 278), (278, 279), (279, 280), (280, 281), (281, 282), (282, 283), (283, 284), (284, 285), (285, 286), (286, 287), (287, 288), (288, 289), (289, 290), (290, 291), (291, 292), (292, 293), (293, 294), (294, 295), (295, 296), (296, 297), (297, 298), (298, 299), (299, 300), (300, 301), (301, 302), (302, 303), (303, 304), (304, 305), (305, 306), (306, 307), (307, 308), (308, 309), (309, 310), (310, 311), (311, 312), (312, 313), (313, 314), (314, 315), (315, 316), (316, 317), (317, 318), (318, 319), (319, 320), (320, 321), (321, 322), (322, 323), (323, 324), (324, 325), (325, 326), (326, 327), (327, 328), (328, 329), (329, 330), (330, 331), (331, 332), (332, 333), (333, 334), (334, 335), (335, 336), (336, 337), (337, 338), (338, 339), (339, 340), (340, 341), (341, 342), (342, 343), (343, 344), (344, 345), (345, 346), (346, 347), (347, 348), (348, 349), (349, 350), (350, 351), (351, 352), (352, 353), (353, 354), (354, 355), (355, 356), (356, 357), (357, 358), (358, 359), (359, 360), (360, 361), (361, 362), (362, 363), (363, 364), (364, 365), (365, 366), (366, 367), (367, 368), (368, 369), (369, 370), (370, 371), (371, 372), (372, 373), (373, 374), (374, 375), (375, 376), (376, 377), (377, 378), (378, 379), (379, 380), (380, 381), (381, 382), (382, 383), (383, 384), (384, 385), (385, 386), (386, 387), (387, 388), (388, 389), (389, 390), (390, 391), (391, 392), (392, 393), (393, 394), (394, 395), (395, 396), (396, 397), (397, 398), (398, 399), (399, 400), (400, 401), (401, 402), (402, 403), (</pre>
--	--

[illegible]

[illegible][illegible]

Example 2:

```

graph LR
    v0((v0)) -- 1 --> v1((v1))
    v0 -- 2 --> v2((v2))
    v1 -- 1 --> v3((v3))
    v1 -- 1 --> v4((v4))
    v2 -- 1 --> v4
    v3 -- 1 --> v5((v5))
    v4 -- 1 --> v5
    v5 -- 1 --> v5
    v5 -- 1 --> v5
    v5 -- 1 --> v6((v6))
    v6 -- 1 --> v7((v7))
    v7 -- 1 --> v8((v8))
    v8 -- 1 --> v9((v9))
  
```

Input:

```

n=10;
m=10;
"v0 v1 1";
"v0 v2 2";
"v1 v3 1";
"v1 v4 1";
"v2 v4 1";
"v3 v5 1";
"v4 v5 1";
"v5 v5 1";
"v5 v5 1";
"v5 v6 1";
"v6 v7 1";
"v7 v8 1";
"v8 v9 1";

```

Output:

```

1 0 0 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 1 0 0 0 0
0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 1

```

Consideration:

- $n = \text{vertices} = 10$
- $m = \text{edges} = 10$
- capacity $c = 1$ for all edges
- $\text{source} = v_0$ and $\text{sink} = v_9$
- $\text{vertices} = \{v_0, v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9\}$
- $\text{edges} = \{v_0 \rightarrow v_1, v_0 \rightarrow v_2, v_1 \rightarrow v_3, v_1 \rightarrow v_4, v_2 \rightarrow v_4, v_3 \rightarrow v_5, v_4 \rightarrow v_5, v_5 \rightarrow v_5, v_5 \rightarrow v_6, v_6 \rightarrow v_7, v_7 \rightarrow v_8, v_8 \rightarrow v_9\}$
- The source may not start or end with the hyper-vertex because $C = 1$.
- You may assume there is no duplicate in set E .

<https://leetcode.com/problems/network-flow-dinics/>

Community Solutions (Python)

```

class Dinic:
    def __init__(self, n):
        self.n = n
        self.g = [[] for _ in range(n)]
        self.level = [0] * n
        self.ptr = [0] * n
        self.flow = 0

    def addEdge(self, u, v, c):
        self.g[u].append((v, c))

    def bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        q = deque([s])
        while q:
            u = q.popleft()
            for v, c in self.g[u]:
                if self.level[v] == -1 and c > 0:
                    self.level[v] = self.level[u] + 1
                    q.append(v)
        return self.level[t] != -1

    def dfs(self, u, t, f):
        if u == t:
            return f
        for i in range(self.ptr[u], len(self.g[u])):
            v, c = self.g[u][i]
            if self.level[v] == self.level[u] + 1 and c > 0:
                f2 = self.dfs(v, t, min(f, c))
                if f2 > 0:
                    self.g[u][i][1] -= f2
                    self.g[v][self.ptr[v] - 1][1] += f2
                    return f2
            self.ptr[u] += 1
        return 0

    def max_flow(self, s, t):
        while self.bfs(s, t):
            f = self.dfs(s, t, float('inf'))
            self.flow += f
        return self.flow

```

• **Wolfram:**

```

n = 10;
m = 10;
"v0 v1 1";
"v0 v2 2";
"v1 v3 1";
"v1 v4 1";
"v2 v4 1";
"v3 v5 1";
"v4 v5 1";
"v5 v5 1";
"v5 v5 1";
"v5 v6 1";
"v6 v7 1";
"v7 v8 1";
"v8 v9 1";

```

001

002

003

004

005

006

007

008

009

010

011

012

013

014

015

016

017

018

019

020

021

022

023

024

025

026

027

028

029

030

031

032

033

034

035

036

037

038

039

040

041

042

043

044

045

046

047

048

049

050

051

052

053

054

055

056

057

058

059

060

061

062

063

064

065

066

067

068

069

070

071

072

073

074

075

076

077

078

079

080

081

082

083

084

085

086

087

088

089

090

091

092

093

094

095

096

097

098

099

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

Figure 1: Example of a network and its corresponding code. The network diagram shows a central 'nodes' node connected to 'edges', which then connects to 'edges2'. 'edges2' is connected to 'edges3', which in turn connects to 'edges4'. 'edges4' is connected to 'edges5', which finally connects to 'edges6'. The code snippet shows a JavaScript-like structure for a network, including a 'nodes' array, an 'edges' array, and a 'links' array. The 'links' array contains a 'source' and a 'target' property, and a 'weight' property. The 'edges' array contains a 'source' and a 'target' property, and a 'weight' property. The 'edges2' array contains a 'source' and a 'target' property, and a 'weight' property. The 'edges3' array contains a 'source' and a 'target' property, and a 'weight' property. The 'edges4' array contains a 'source' and a 'target' property, and a 'weight' property. The 'edges5' array contains a 'source' and a 'target' property, and a 'weight' property. The 'edges6' array contains a 'source' and a 'target' property, and a 'weight' property.

[illegible][illegible][illegible]

001	0.0007	001	0.0008	001	0.0008	001
new get	<pre> self.set_self_return_value() # If the link is not being cloned is indicated on the thread, # generate the links. this is 10 cases # If the thread is not being cloned, then the cloning is # done by a modified thread. In this case we are using # the thread object, thread = Thread(target=self.get_self_return_value) # If we are cloning we will using a self.knowledge. # instance = self.knowledge, thread = Thread(target=self.get_self_return_value) # If this thread constructs a link, "thread" will be # the new get and other stuff. This case constructs the # link at a particular the generation step, the network # function will be used to generate the links. # If it is not a thread, then the thread will be a thread. </pre>	<pre> link = self.get_self_return_value() # If we are cloning the thread, then the thread is # the thread object, thread = Thread(target=self.get_self_return_value) # If we are cloning we will using a self.knowledge. # instance = self.knowledge, thread = Thread(target=self.get_self_return_value) # If this thread constructs a link, "thread" will be # the new get and other stuff. This case constructs the # link at a particular the generation step, the network # function will be used to generate the links. # If it is not a thread, then the thread will be a thread. </pre>	<pre> print(link.get_self_return_value()) # If the thread is not being cloned, then the cloning is # done by a modified thread. In this case we are using # the thread object, thread = Thread(target=self.get_self_return_value) # If we are cloning we will using a self.knowledge. # instance = self.knowledge, thread = Thread(target=self.get_self_return_value) # If this thread constructs a link, "thread" will be # the new get and other stuff. This case constructs the # link at a particular the generation step, the network # function will be used to generate the links. # If it is not a thread, then the thread will be a thread. </pre>	<pre> print(link.get_self_return_value()) # If the thread is not being cloned, then the cloning is # done by a modified thread. In this case we are using # the thread object, thread = Thread(target=self.get_self_return_value) # If we are cloning we will using a self.knowledge. # instance = self.knowledge, thread = Thread(target=self.get_self_return_value) # If this thread constructs a link, "thread" will be # the new get and other stuff. This case constructs the # link at a particular the generation step, the network # function will be used to generate the links. # If it is not a thread, then the thread will be a thread. </pre>	<pre> print(link.get_self_return_value()) # If the thread is not being cloned, then the cloning is # done by a modified thread. In this case we are using # the thread object, thread = Thread(target=self.get_self_return_value) # If we are cloning we will using a self.knowledge. # instance = self.knowledge, thread = Thread(target=self.get_self_return_value) # If this thread constructs a link, "thread" will be # the new get and other stuff. This case constructs the # link at a particular the generation step, the network # function will be used to generate the links. # If it is not a thread, then the thread will be a thread. </pre>	

p228		p227	p25
9269 Alien Dictionary hard There is a list of <i>N</i> strings <i>s</i> representing words in an alien language that is based on the alphabet <i>English</i>. Write a method to find the order of the alphabet (i.e., it should sort the given words of the list by comparing them in ascending order). Time Complexity: $O(N \times L)$, where N is the number of words, L is the average length of words. $O(N \times L)$ is the number of string characters, and L is the length of the longest word. Problem There is a new alien language that uses the English alphabet. However, the order of the letters is unknown to you. Given a list of strings <i>s</i> representing words in the alien language, you are to find the order of the letters. The strings in <i>s</i> may be sorted lexicographically by order of the letters, or they may not. In the latter case, you are to return an empty string. The input strings <i>s</i> will be given in an order that may be sorted lexicographically by order of the letters, or they may not. In the latter case, you are to return an empty string. Example 1: <pre>Input: ["wrt","kcp","lqn"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 2: <pre>Input: ["z","x","c","b","a"] Output: "zxcba" Explanation: The order is [z,x,c,b,a], so return "".</pre> Example 3: <pre>Input: ["wrt","kcp","lqn","t"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 4: <pre>Input: ["wrt","kcp","lqn","t","y"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 5: <pre>Input: ["wrt","kcp","lqn","t","y","z"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 6: <pre>Input: ["wrt","kcp","lqn","t","y","z","x"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 7: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 8: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 9: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 10: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 11: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 12: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 13: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 14: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 15: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 16: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 17: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 18: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 19: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 20: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 21: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 22: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 23: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 24: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 25: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 26: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 27: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 28: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 29: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 30: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w","x"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 31: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w","x","y"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 32: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w","x","y","z"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 33: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w","x","y","z","a"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 34: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w","x","y","z","a","b"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 35: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w","x","y","z","a","b","c"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 36: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w","x","y","z","a","b","c","d"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 37: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w","x","y","z","a","b","c","d","e"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 38: <pre>Input: ["wrt","kcp","lqn","t","y","z","x","a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v","w","x","y","z","a","b","c","d","e","f"] Output: "wklqncp" Explanation: The order is [w,k,l,q,n,c,p], so return "".</pre> Example 			

0%	25%	50%	75%	100%
<pre> class Solution: def shortestPath(self, words: List[str], target: str) -> int: n = len(words) adj = [[] for _ in range(n)] for i, word in enumerate(words): for j, otherWord in enumerate(words): if i < j and self.isSimilar(word, otherWord): adj[i].append(j) adj[j].append(i) return self.bfs(words[0], target, adj) def isSimilar(self, word1: str, word2: str) -> bool: diff = 0 for i in range(len(word1)): if word1[i] != word2[i]: diff += 1 if diff > 2: return False return diff == 2 def bfs(self, start: str, target: str, adj: List[List[int]]) -> int: queue = deque([start]) visited = set([start]) while queue: word = queue.popleft() if word == target: return 1 for neighbor in adj[word]: if neighbor not in visited: queue.append(neighbor) visited.add(neighbor) return -1 </pre>	<pre> def findWords(words: List[str], target: str) -> List[str]: words = sorted(words, key=len) res = [] for word in words: if len(word) < len(target): continue if word[:len(target)] == target: res.append(word) return res </pre>	<pre> def findWords(words: List[str], target: str) -> List[str]: words = sorted(words, key=len) res = [] for word in words: if len(word) < len(target): continue if word[:len(target)] == target: res.append(word) return res </pre>	<pre> def findWords(words: List[str], target: str) -> List[str]: words = sorted(words, key=len) res = [] for word in words: if len(word) < len(target): continue if word[:len(target)] == target: res.append(word) return res </pre>	<pre> def findWords(words: List[str], target: str) -> List[str]: words = sorted(words, key=len) res = [] for word in words: if len(word) < len(target): continue if word[:len(target)] == target: res.append(word) return res </pre>

[illegible][illegible][illegible]

<pre> def dfs(x, y): if x < 0 or y < 0 or x > len(grid) - 1 or y > len(grid) - 1 or visited[x][y]: return 0 visited[x][y] = 1 up = dfs(x, y - 1) down = dfs(x, y + 1) left = dfs(x - 1, y) right = dfs(x + 1, y) return 1 + max(up, down, left, right) visited = [[0] * len(grid[0]) for _ in range(len(grid))] return dfs(0, 0) </pre>	<pre> def longestIncreasingPath(grid): if not grid: return 0 m, n = len(grid), len(grid[0]) res = 0 for i in range(m): for j in range(n): res = max(res, dfs(i, j, grid)) return res def dfs(i, j, grid): if i < 0 or i > m - 1 or j < 0 or j > n - 1 or visited[i][j]: return 0 visited[i][j] = 1 up = dfs(i - 1, j, grid) down = dfs(i + 1, j, grid) left = dfs(i, j - 1, grid) right = dfs(i, j + 1, grid) return 1 + max(up, down, left, right) </pre>	<pre> def longestIncreasingPath(grid): if not grid: return 0 m, n = len(grid), len(grid[0]) res = 0 for i in range(m): for j in range(n): res = max(res, dfs(i, j, grid)) return res def dfs(i, j, grid): if i < 0 or i > m - 1 or j < 0 or j > n - 1 or visited[i][j]: return 0 visited[i][j] = 1 up = dfs(i - 1, j, grid) down = dfs(i + 1, j, grid) left = dfs(i, j - 1, grid) right = dfs(i, j + 1, grid) return 1 + max(up, down, left, right) </pre>	912 329 Longest Increasing Path in a Matrix LeetCode Simplex Medium LeetCode Given an $m \times n$ matrix of integers, find the longest increasing path within the matrix. You may move up, down, left, or right from grid cell (i, j) to grid cell (x, y) if and only if $grid[i][j] < grid[x][y]$. Your function should return the length of the longest path. Example 1: <pre> [[9,9,4], [5,6,4], [1,1,6]] </pre> Return 6. The longest increasing path is $1 \rightarrow 6 \rightarrow 4 \rightarrow 6 \rightarrow 5 \rightarrow 9$. Example 2: <pre> [[1,2,3,4,5], [4,3,2,1,0], [3,2,1,0,9], [2,1,0,9,8], [1,0,9,8,7]] </pre> Return 10. The longest increasing path is $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10$.
--	---	---	--

[illegible][illegible]

DP1: DP1 = DP1 + DP2

DP2: DP2 = DP2 + DP3

DP3: DP3 = DP3 + DP4

Comments:

```

n = len(matrix)
m = len(matrix[0])
# 0 < i < m, 0 < j < n
# 0 < matrix[i][j] <= 255 - 1
# Community Solution (Python)
def solve():
    # DP table
    dp = [[0]*m for _ in range(n)]
    dp[0][0] = matrix[0][0]
    # DP table
    for i in range(1, n):
        for j in range(1, m):
            dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + matrix[i][j]
    return dp[n-1][m-1]

```

a22563

where u is a parent of u child.

Remove an edge that can be removed so that the resulting graph is a rooted tree of n nodes. If there are multiple answers, return the answer that occurs earliest in the 2D-array.

Example 1:

```
graph TD
    1((1)) --> 2((2))
    1((1)) --> 3((3))
    2((2)) --> 3((3))
```

Input: edges = [[1,2], [1,3], [2,3]]
Output: [1, 2]

Example 2:

```
graph TD
    1((1)) --> 2((2))
    1((1)) --> 3((3))
    2((2)) --> 4((4))
    3((3)) --> 4((4))
```

Input: edges = [[1,2], [1,3], [2,4], [3,4]]
Output: [1, 3]

a22564

is a pair (u, v) that represents a directed edge connecting nodes u and v .

Example 1:

```

n = 5; edges = [[1,2], [1,3], [2,4], [3,4]]
return edges - 2
// [1, 3]
n = 4; edges = [[1,2], [1,3], [2,4]]
return [1, 4]

```

Input Format (Python):

- 1** Write Python Script

2 Write Python Script

After clicking on the button, you will see the following code editor:

```

def removeRedundantEdges(edges):
    # Write your code here
    # Return: List of 2D array containing edges
    # Example:
    # Input: edges = [[1,2], [1,3], [2,4], [3,4]]
    # Output: [1, 3]
    # If no edge can be removed, return []

```

For more sample inputs, click on the button:

a22565

DP1	g2248	DP1	Sheet 250 / 265 - LevMasonry By Pattern - Python Only - 9-up Castings	g2249	DP1	g2250
-----	-------	-----	---	-------	-----	-------

Input: matrix = [[1]] Output: 1	Sheet 251 / 265 · LearnMastery By Problem - Python Only - 9-up 0.2218	Sheet 252 / 265 · LearnMastery By Problem - Python Only - 9-up 0.2219	Sheet 253 / 265 · LearnMastery By Problem - Python Only - 9-up 0.2220
---	---	---	---

	<pre> def __init__(self, *args, **kwargs): self.args = args self.kwargs = kwargs self.constraints = {} for constraint in self.constraints: self.constraints[constraint] = self.constraints[constraint]() return cast </pre>	
3	p2266 DP% Sheet 252 / 265 - LeeMastery By Pattern - Python Only - 9-up	p2267 DP%

[illegible]

[illegible]

[illegible]

[illegible]

```

class Solution {
public:
    int sumOfSquares(vector<int>& nums) {
        int sum = 0;
        for (int i = 0; i < nums.size(); i++) {
            sum += nums[i] * nums[i];
        }
        return sum;
    }
};

```

```

class Solution {
public:
    int sumOfSquares(vector<int>& nums) {
        int sum = 0;
        for (int i = 0; i < nums.size(); i++) {
            sum += nums[i] * nums[i];
        }
        return sum;
    }
};

```

```

class Solution {
public:
    int sumOfSquares(vector<int>& nums) {
        int sum = 0;
        for (int i = 0; i < nums.size(); i++) {
            sum += nums[i] * nums[i];
        }
        return sum;
    }
};

```

• WRACC			
<pre> class Solution: def decomposingInteger(self, n: int) -> int: ans = 0 while n > 0: ans += (n // 10) * 10 n = n // 10 return ans </pre>	<pre> class Solution: def decomposingInteger(self, n: int) -> int: ans = 0 while n > 0: ans += (n // 10) * 10 n = n // 10 return ans </pre>	<pre> class Solution: def decomposingInteger(self, n: int) -> int: ans = 0 while n > 0: ans += (n // 10) * 10 n = n // 10 return ans </pre>	<pre> class Solution: def decomposingInteger(self, n: int) -> int: ans = 0 while n > 0: ans += (n // 10) * 10 n = n // 10 return ans </pre>
<pre> class Solution: def decomposingInteger(self, n: int) -> int: ans = 0 while n > 0: ans += (n // 10) * 10 n = n // 10 return ans </pre>	<pre> class Solution: def decomposingInteger(self, n: int) -> int: ans = 0 while n > 0: ans += (n // 10) * 10 n = n // 10 return ans </pre>	<pre> class Solution: def decomposingInteger(self, n: int) -> int: ans = 0 while n > 0: ans += (n // 10) * 10 n = n // 10 return ans </pre>	<pre> class Solution: def decomposingInteger(self, n: int) -> int: ans = 0 while n > 0: ans += (n // 10) * 10 n = n // 10 return ans </pre>

task	p200a	task	p200b	task
<pre> 4: 5: <code> 6: <pre> 7: </pre> 8: </code> 9: </pre> </pre>	<pre> 1: <code> 2: <pre> 3: </pre> 4: </code> 5: </pre> </pre>	<pre> 1: <code> 2: <pre> 3: </pre> 4: </code> 5: </pre> </pre>	<pre> 1: <code> 2: <pre> 3: </pre> 4: </code> 5: </pre> </pre>	<pre> 1: <code> 2: <pre> 3: </pre> 4: </code> 5: </pre> </pre>

[illegible]

task	p2014	task	p2015	task	p2016
<pre> def isPrime(n): """ Returns True if n is prime, False if n is not prime. """ if n < 2: return False for i in range(2, n): if n % i == 0: return False return True def isPrime(n): """ Returns True if n is prime, False if n is not prime. """ if n < 2: return False for i in range(2, n): if n % i == 0: return False return True </pre>	<pre> def isPrime(n): """ Returns True if n is prime, False if n is not prime. """ if n < 2: return False for i in range(2, n): if n % i == 0: return False return True def isPrime(n): """ Returns True if n is prime, False if n is not prime. """ if n < 2: return False for i in range(2, n): if n % i == 0: return False return True </pre>	<pre> def isPrime(n): """ Returns True if n is prime, False if n is not prime. """ if n < 2: return False for i in range(2, n): if n % i == 0: return False return True def isPrime(n): """ Returns True if n is prime, False if n is not prime. """ if n < 2: return False for i in range(2, n): if n % i == 0: return False return True </pre>	<pre> def isPrime(n): """ Returns True if n is prime, False if n is not prime. """ if n < 2: return False for i in range(2, n): if n % i == 0: return False return True def isPrime(n): """ Returns True if n is prime, False if n is not prime. """ if n < 2: return False for i in range(2, n): if n % i == 0: return False return True </pre>		

[illegible][illegible]

Task	p.2057	Task	p.2058	Task	p.2059
4039 Tree Expression Parser medium – GoogleMap – WCCC – Local Sorting, Stack, DFS		Example 5: <pre>Input: expression = "1+2*3" Output: 7 Explanation: 1+2*3 = 1+(2*3) = 1+6 = 7.</pre>	Example 6: <pre>Input: expression = "1+2*3+4" Output: 10 Explanation: 1+2*3+4 = 1+(2*3)+4 = 1+6+4 = 11.</pre>	Example 7: <pre>Input: expression = "1+2*3+4*5" Output: 22 Explanation: 1+2*3+4*5 = 1+(2*3)+(4*5) = 1+6+20 = 27.</pre>	
Problem Given a string expression which contains a valid arithmetic expression, evaluate the expression, and return the result of it. The expression contains the operators '+', '-', '*', and '/'. The expression contains the operands '1' through '9'. The expression contains the parentheses '(', ')', and the space character ' '. The expression contains the operators '+', '-', '*', and '/'. The expression contains the operands '1' through '9'. The expression contains the parentheses '(', ')', and the space character ' '. The expression contains the operators '+', '-', '*', and '/'. The expression contains the operands '1' through '9'. The expression contains the parentheses '(', ')', and the space character ' '.	Example 1: <pre>Input: expression = "1+2*3" Output: 7 Explanation: 1+2*3 = 1+(2*3) = 1+6 = 7.</pre>	Example 2: <pre>Input: expression = "1+2*3+4" Output: 10 Explanation: 1+2*3+4 = 1+(2*3)+4 = 1+6+4 = 11.</pre>	Example 3: <pre>Input: expression = "1+2*3+4*5" Output: 22 Explanation: 1+2*3+4*5 = 1+(2*3)+(4*5) = 1+6+20 = 27.</pre>	Example 4: <pre>Input: expression = "1+2*3+4*5*6" Output: 78 Explanation: 1+2*3+4*5*6 = 1+(2*3)+(4*5*6) = 1+6+120 = 127.</pre>	

Task	prEREQ	Task	prEREQ	Task	prEREQ
<pre> + ListIndex class ListIndex { public: ListIndex() {} ListIndex(int n): mSize(n), mExpression("") {} ListIndex(int n, const string& exp): mSize(n), mExpression(exp) {} ~ListIndex() {} // Getters int GetSize() const { return mSize; } const string& GetExpression() const { return mExpression; } // Setters void SetSize(int n) { mSize = n; } void SetExpression(const string& exp) { mExpression = exp; } // Methods void AddElement(int value) { mElements.push_back(value); } void RemoveElement(int value) { mElements.erase(remove(mElements.begin(), mElements.end(), value), mElements.end()); } void Print() const { cout << "ListIndex: [" << mElements << "]" << endl; } // Private attributes int mSize; string mExpression; vector<int> mElements; }; </pre>		<pre> + SimplifyExp class SimplifyExp { public: SimplifyExp() {} SimplifyExp(const string& exp): mExpression(exp) {} ~SimplifyExp() {} // Getters const string& GetExpression() const { return mExpression; } // Setters void SetExpression(const string& exp) { mExpression = exp; } // Methods string Simplify() const { string result = mExpression; // Simplify the expression // ... (simplification logic) ... return result; } // Private attributes string mExpression; }; </pre>		<pre> + DigitSum class DigitSum { public: DigitSum() {} DigitSum(int n): mNumber(n) {} ~DigitSum() {} // Getters int GetNumber() const { return mNumber; } // Setters void SetNumber(int n) { mNumber = n; } // Methods int SumOfDigits() const { int sum = 0; while (mNumber > 0) { sum += mNumber % 10; mNumber /= 10; } return sum; } // Private attributes int mNumber; }; </pre>	

[illegible]

```

graph LR
    subgraph Stack
        Temp[Temp]
        TempArray[TempArray]
    end
    Temp --> TempArray
    TempArray --> Temp
  
```

The diagram illustrates the flow of data between a **Stack** and a **Temp** variable. The **Stack** contains a **Temp** variable and a **TempArray** array. The **Temp** variable is updated with the value of **TempArray[0]**. The **TempArray** array is then updated with the value of **Temp**.

Task	Difficulty	Track	Difficulty	Track
Given an array of numbers, find the maximum sum of a subarray. <pre> def maxSubArray(nums): # Write your code here # Return the maximum sum of a subarray # Example: # Input: [-2, 1, -3, 4, -1, 2, 1, -4, 4] # Output: 6 # Explanation: The subarray [4, -1, 2, 1] has the maximum sum of 6. </pre>	Easy	Track Find	Given an array of numbers, find the maximum sum of a subarray. <pre> def maxSubArray(nums): # Write your code here # Return the maximum sum of a subarray # Example: # Input: [-2, 1, -3, 4, -1, 2, 1, -4, 4] # Output: 6 # Explanation: The subarray [4, -1, 2, 1] has the maximum sum of 6. </pre>	Track Find
Given an array of numbers, find the maximum sum of a subarray. <pre> def maxSubArray(nums): # Write your code here # Return the maximum sum of a subarray # Example: # Input: [-2, 1, -3, 4, -1, 2, 1, -4, 4] # Output: 6 # Explanation: The subarray [4, -1, 2, 1] has the maximum sum of 6. </pre>	Easy	Track Find	Given an array of numbers, find the maximum sum of a subarray. <pre> def maxSubArray(nums): # Write your code here # Return the maximum sum of a subarray # Example: # Input: [-2, 1, -3, 4, -1, 2, 1, -4, 4] # Output: 6 # Explanation: The subarray [4, -1, 2, 1] has the maximum sum of 6. </pre>	Track Find

[illegible][illegible][illegible]

Stack <pre> return count; } Community Services (Python) #BASIC def findMedian(self, nums): """ :type nums: List[int] :rtype: float """ # Sort the array first n = len(nums) if n % 2 == 1: # Odd number of elements return float(nums[n // 2]) else: # Even number of elements return float((nums[n // 2 - 1] + nums[n // 2]) / 2) </pre>	Stack <pre> class Stack: def __init__(self): self.items = [] def push(self, item): self.items.append(item) def pop(self): if not self.is_empty(): return self.items.pop() return None def is_empty(self): return len(self.items) == 0 </pre>	Stack <pre> def findMedian(self, nums): """ :type nums: List[int] :rtype: float """ return self._findMedian(nums) def _findMedian(self, nums): """ :type nums: List[int] :rtype: float """ # Sort the array first n = len(nums) if n % 2 == 1: # Odd number of elements return float(nums[n // 2]) else: # Even number of elements return float((nums[n // 2 - 1] + nums[n // 2]) / 2) </pre>	Stack <pre> def findMedian(self, nums): """ :type nums: List[int] :rtype: float """ return self._findMedian(nums) def _findMedian(self, nums): """ :type nums: List[int] :rtype: float """ # Sort the array first n = len(nums) if n % 2 == 1: # Odd number of elements return float(nums[n // 2]) else: # Even number of elements return float((nums[n // 2 - 1] + nums[n // 2]) / 2) </pre>
---	--	---	---

Snack	g2002	Snack	g2002	Snack	g2002
<pre> class Solution: def getTemperature(self, temperatures: List[int]) -> List[int]: n = len(temperatures) ans = [0] * n for i in range(n): # find the index of the first temperature that is higher than the current one for j in range(i+1, n): if temperatures[j] > temperatures[i]: ans[i] = j - i break else: ans[i] = 0 return ans </pre>		<pre> class Solution: def getTemperature(self, temperatures: List[int]) -> List[int]: n = len(temperatures) ans = [0] * n for i in range(n): # find the index of the first temperature that is higher than the current one for j in range(i+1, n): if temperatures[j] > temperatures[i]: ans[i] = j - i break else: ans[i] = 0 return ans </pre>		<pre> class Solution: def getTemperature(self, temperatures: List[int]) -> List[int]: n = len(temperatures) ans = [0] * n for i in range(n): # find the index of the first temperature that is higher than the current one for j in range(i+1, n): if temperatures[j] > temperatures[i]: ans[i] = j - i break else: ans[i] = 0 return ans </pre>	
• Temperature		• Temperature		• Temperature	

[illegible][illegible]

Book	Page#	Book	Page#	Book	Page#
Input: heights = [3,8,5,1] Output: 6 Explanation: In this example, the only building that can be seen from across the horizon is building 3 (the second) since it has no other taller buildings behind it. Example 2: Input: heights = [3,8,5,1] Output: 1 Explanation: In this example, the only building that can be seen from across the horizon is building 1 (the first). Input: heights = [3,8,5,1] Output: 1 Explanation: Only building 1 has no other taller buildings behind it. Constraints: 0 < heights.length <= 100 0 < heights[i] <= 100 Brute Force (Python)					
• Brute Force Approach <pre>class Solution: def canSeeFromSea(self, heights: List[int]) -> int: n = len(heights) # If there is only 1 building, it can be seen from the ocean. if n == 1: return 1 # Loop through each building and check if it can be seen. count = 0 for i in range(n): # If the current building is taller than all buildings behind it, it can be seen. if heights[i] > heights[i-1]: count += 1 return count</pre> Complexity: O(n)					
• Monotonic Stack (Python) <pre>class Solution: def canSeeFromSea(self, heights: List[int]) -> int: n = len(heights) # If there is only 1 building, it can be seen from the ocean. if n == 1: return 1 # Loop through each building and check if it can be seen. count = 0 for i in range(n): # If the current building is taller than all buildings behind it, it can be seen. if heights[i] > heights[i-1]: count += 1 return count</pre> Complexity: O(n)					

Stack	Stack	Stack	Stack
<pre> class Solution: def minOperations(self, nums: List[int]) -> int: ans = 0 for i in range(len(nums)): if nums[i] % 2 == 1: ans += 1 return ans </pre> • Simplified	<pre> def minOperations(nums: List[int]) -> int: return sum(1 for num in nums if num % 2) </pre>	<pre> # Python solution for Russian Math Prep def minOperations(nums: List[int]) -> int: """ Count the number of odd integers in the array. For each odd integer, we need to perform one operation to make it even. """ return sum(1 for num in nums if num % 2) </pre>	<pre> class Solution: def minOperations(self, nums: List[int]) -> int: ans = 0 for i in range(len(nums)): if nums[i] % 2 == 1: ans += 1 return ans </pre> • Simplified

2000	Stack	p2000	Stack	p2000	Stack	p2000
	<pre>def testRight(Threshold, i, test): # If the value is less than the threshold, return 0 if test < Threshold: return 0 # If the value is greater than the threshold, return 1 else: return 1</pre>	<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val): self.val = val self.left = None self.right = None</pre>		<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val): self.val = val self.left = None self.right = None</pre>	<pre>def testRight(test): # Return function to traverse second tree and check for complete # and BST property return test</pre>	
	<pre>def testLeft(Threshold, i, test): # If the value is less than the threshold, return 0 if test < Threshold: return 0 # If the value is greater than the threshold, return 1 else: return 1</pre>	<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val): self.val = val self.left = None self.right = None</pre>	<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val): self.val = val self.left = None self.right = None</pre>	<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val): self.val = val self.left = None self.right = None</pre>	<pre>def testLeft(test): # Return function to traverse second tree and check for complete # and BST property return test</pre>	
	<pre>def testBoth(Threshold, i, test): # If the value is less than the threshold, return 0 if test < Threshold: return 0 # If the value is greater than the threshold, return 1 else: return 1</pre>	<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val): self.val = val self.left = None self.right = None</pre>	<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val): self.val = val self.left = None self.right = None</pre>	<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val): self.val = val self.left = None self.right = None</pre>	<pre>def testBoth(test): # Return function to traverse second tree and check for complete # and BST property return test</pre>	

a2019	a2019	a2019	a2019
<pre> def isWellFormed(s: String): Boolean = { def isWellFormed(s: String, stack: List[String]): Boolean = { if (s.isEmpty) stack.isEmpty else if (s.head == '(') isWellFormed(s.tail, stack :: List()) else if (s.head == ')') !stack.isEmpty && isWellFormed(s.tail, stack.drop(1)) else false } isWellFormed(s, List()) } </pre>	<pre> def isWellFormed(s: String): Boolean = { def isWellFormed(s: String, stack: List[String]): Boolean = { if (s.isEmpty) stack.isEmpty else if (s.head == '(') isWellFormed(s.tail, stack :: List()) else if (s.head == ')') !stack.isEmpty && isWellFormed(s.tail, stack.drop(1)) else false } isWellFormed(s, List()) } </pre>	Stack #32 Longest Valid Parentheses LeetCode's "Longest Valid" / "Longest" using "Dynamic Programming" solution. Link: https://leetcode.com/problems/longest-valid-parentheses/ Time Complexity: $O(n^2)$ where n is the length of the input. Space Complexity: $O(n)$ due to the stack data structure. Problem: Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring. Example 1: <pre> Input: s = "(()" Output: 2 Explanation: The longest valid parentheses substring is "()". </pre>	Stack #32 Longest Valid Parentheses LeetCode's "Longest Valid" / "Longest" using "Dynamic Programming" solution. Link: https://leetcode.com/problems/longest-valid-parentheses/ Time Complexity: $O(n^2)$ where n is the length of the input. Space Complexity: $O(n)$ due to the stack data structure. Problem: Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring. Example 1: <pre> Input: s = "(()" Output: 2 Explanation: The longest valid parentheses substring is "()". </pre>

[illegible]

a20752	a20752	a20752	a20752
<pre># initialize for a binary tree node. def __init__(self, val=None, left=None, right=None): self.val = val self.left = left self.right = right def insert(self, root, val): if not root: return Node(val) if val < root.val: root.left = insert(root.left, val) else: root.right = insert(root.right, val) return root</pre>	<pre>1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778</pre>		

[illegible]

[illegible]

419. Sliding Window Maximum [Hard] Class Problem Given an array <code>nums</code> of size <code>n</code> and a window size <code>k</code>, return the maximum value for each window of size <code>k</code> as it slides from left to right. Key Insights - Use a deque to maintain the indices of elements in the current window. - Remove elements from the back if they are smaller than the current element. - Remove elements from the front if they are out of the current window. - The front of the deque always contains the maximum element of the current window. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(k)$	<pre>def maxSlidingWindow(nums: List[int], k: int) -> List[int]: if not nums: return [] n = len(nums) if k > n: return nums deque = collections.deque() result = [] for i in range(n): # Remove elements out of the window while deque and deque[0] <= i - k: deque.popleft() # Remove elements smaller than the current element while deque and nums[deque[-1]] < nums[i]: deque.pop() deque.append(i) # Add the maximum element to the result if i >= k - 1: result.append(nums[deque[0]]) return result</pre>	420. Strong Password Checker [Hard] Class Problem Given a password <code>password</code>, return the minimum number of steps to make the password strong. A password is considered strong if it has a length of at least 6, contains at least one lowercase letter, one uppercase letter, one digit, and one special character. Key Insights - Calculate the number of missing characters for each category. - Calculate the number of steps to fix the password based on the length and the number of missing characters. Space & Time Time Complexity: $O(1)$ Space Complexity: $O(1)$	<pre>def strongPasswordChecker(password: str) -> int: n = len(password) missing = [0, 0, 0, 0] for c in password: if c.islower(): missing[0] -= 1 elif c.isupper(): missing[1] -= 1 elif c.isdigit(): missing[2] -= 1 elif c in "!@#\$%^&*()-_+=": missing[3] -= 1 missing = [max(0, x) for x in missing] steps = 0 if n < 6: steps += 6 - n elif n > 6: steps += (n - 6) // 3 for i in range(2, -1, -1): steps += min(missing[i], (n - 6) // 3) missing[i] -= min(missing[i], (n - 6) // 3) steps += sum(missing) return steps</pre>	421. Maximum XOR of Two Numbers in an Array [Medium] Class Problem Given an array of numbers, return the maximum value of <code>x XOR y</code> where <code>x</code> and <code>y</code> are elements of the array. Key Insights - Use a trie to store the binary representation of the numbers. - For each number, find the maximum XOR by traversing the trie. Space & Time Time Complexity: $O(n \cdot \log \text{max_value})$ Space Complexity: $O(\log \text{max_value})$	<pre>def findMaximumXOR(nums: List[int]) -> int: if not nums: return 0 n = len(nums) max_xor = 0 for i in range(n): for j in range(i + 1, n): xor = nums[i] ^ nums[j] max_xor = max(max_xor, xor) return max_xor</pre>	422. Valid Word Square [Medium] Class Problem Given a list of words, return <code>true</code> if they form a valid word square, where the words can be read row-wise and column-wise. Key Insights - Check if the words are of the same length. - Check if the words can be read row-wise and column-wise. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(1)$	<pre>def validWordSquare(words: List[str]) -> bool: n = len(words) for i in range(n): for j in range(n): if j > len(words[i]): return False if words[i][j] != words[j][i]: return False return True</pre>	423. Reconstruct Itinerary [Hard] Class Problem Given a list of tickets, return the itinerary that starts from <code>"JFK"</code> and visits all cities exactly once. Key Insights - Use a priority queue to sort the tickets by destination. - Use a recursive function to find the itinerary. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(n)$	<pre>def findItinerary(tickets: List[List[str]]) -> List[str]: graph = defaultdict(list) for src, dst in tickets: graph[src].append(dst) for src in graph: graph[src].sort() itinerary = [] def dfs(src): while graph[src]: dst = graph[src].pop(0) dfs(dst) itinerary.append(src) dfs("JFK") return itinerary[::-1]</pre>											
424. Longest Repeating Substring [Medium] Class Problem Given a string <code>s</code>, return the length of the longest substring that appears at least twice. Key Insights - Use a sliding window to find the longest repeating substring. - Use a hash map to store the frequency of substrings. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(n)$	<pre>def longestRepeatingSubstring(s: str) -> int: n = len(s) max_length = 0 for i in range(n): for j in range(i + 1, n): sub = s[i:j] count = s.count(sub) if count > 1: max_length = max(max_length, len(sub)) return max_length</pre>	425. Valid Word Square II [Medium] Class Problem Given a list of words, return <code>true</code> if they form a valid word square, where the words can be read row-wise and column-wise. Key Insights - Check if the words are of the same length. - Check if the words can be read row-wise and column-wise. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(1)$	<pre>def validWordSquare(words: List[str]) -> bool: n = len(words) for i in range(n): for j in range(n): if j > len(words[i]): return False if words[i][j] != words[j][i]: return False return True</pre>	426. Inverse Pairs [Medium] Class Problem Given an array of numbers, return the number of inverse pairs. Key Insights Use a merge sort algorithm to count the inverse pairs. Space & Time Time Complexity: $O(n \log n)$ Space Complexity: $O(n)$	<pre>def inversePairs(nums: List[int]) -> int: def mergeSort(nums): if len(nums) <= 1: return nums, 0 mid = len(nums) // 2 left, left_pairs = mergeSort(nums[:mid]) right, right_pairs = mergeSort(nums[mid:]) merged, merge_pairs = merge(left, right) return merged, left_pairs + right_pairs + merge_pairs _, pairs = mergeSort(nums) return pairs</pre>	427. Constructive String [Medium] Class Problem Given a string <code>s</code>, return <code>true</code> if it is a constructive string, where the string can be formed by concatenating the characters of the string. Key Insights Use a recursive function to check if the string is constructive. Space & Time Time Complexity: $O(2^n)$ Space Complexity: $O(n)$	<pre>def isConstructive(s: str) -> bool: def dfs(i): if i == len(s): return True for j in range(i + 1, len(s) + 1): if s[i:j] in words: if dfs(j): return True return False words = ["a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m", "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z"] return dfs(0)</pre>	428. Longest Palindromic Substring [Medium] Class Problem Given a string <code>s</code>, return the longest palindromic substring. Key Insights Use a dynamic programming approach to find the longest palindromic substring. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(n^2)$	<pre>def longestPalindrome(s: str) -> str: n = len(s) dp = [[0] * n for _ in range(n)] for i in range(n): dp[i][i] = 1 for i in range(n - 1, -1, -1): for j in range(i + 1, n): if s[i] == s[j]: dp[i][j] = dp[i + 1][j - 1] + 1 else: dp[i][j] = 0 start, end = 0, 0 for i in range(n): for j in range(n): if dp[i][j] > dp[start][end]: start, end = i, j return s[start:end + 1]</pre>											
429. N-Queens II [Hard] Class Problem Given an integer <code>n</code>, return all possible <code>n</code>-queens solutions. Key Insights Use a backtracking algorithm to find all possible solutions. Space & Time Time Complexity: $O(n!)$ Space Complexity: $O(n)$	<pre>def solveNQueens(n: int) -> List[List[str]]: def dfs(row): if row == n: result.append(current) return for col in range(n): if not isSafe(row, col): continue current.append(col) dfs(row + 1) current.pop() result = [] dfs(0) return result</pre>	430. Flatten Binary Tree to Linked List [Medium] Class Problem Given a binary tree, flatten it to a linked list in-place. Key Insights Use a recursive function to flatten the tree. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(1)$	<pre>def flatten(root: Optional[TreeNode]) -> None: def dfs(node): if not node: return dfs(node.left) node.left = None node.right = dfs(node.right) dfs(root)</pre>	431. Encode String with Shortest Length [Medium] Class Problem Given a string <code>s</code>, return the shortest length of an encoded string. Key Insights Use a dynamic programming approach to find the shortest length. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(n^2)$	<pre>def findLengthEncoding(s: str) -> int: n = len(s) dp = [[0] * n for _ in range(n)] for i in range(n): dp[i][i] = 1 for i in range(n - 1, -1, -1): for j in range(i + 1, n): if s[i] == s[j]: dp[i][j] = dp[i + 1][j - 1] + 1 else: dp[i][j] = 0 return dp[0][n - 1]</pre>	432. All Opts [Medium] Class Problem Given an array of numbers, return the maximum number of operations you can perform. Key Insights Use a greedy algorithm to maximize the number of operations. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(1)$	<pre>def maxOperations(nums: List[int]) -> int: n = len(nums) count = 0 for i in range(n): if nums[i] < 1: continue count += 1 nums[i] -= 1 return count</pre>	433. Minimum Flips to Make a Binary String Monotonic [Medium] Class Problem Given a binary string <code>s</code>, return the minimum number of flips to make it monotonic. Key Insights Use a dynamic programming approach to find the minimum number of flips. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(1)$	<pre>def minFlips(s: str) -> int: n = len(s) dp0 = 0 dp1 = 0 for i in range(n): if s[i] == '0': dp0, dp1 = dp1, dp0 + 1 else: dp0, dp1 = dp0 + 1, dp1 return min(dp0, dp1)</pre>	434. Number of Digits in Hexadecimal [Medium] Class Problem Given a number <code>n</code>, return the number of digits in its hexadecimal representation. Key Insights Use a recursive function to count the number of digits. Space & Time Time Complexity: $O(\log n)$ Space Complexity: $O(1)$	<pre>def hexadecimalLength(n: int) -> int: if n < 16: return 1 return 1 + hexadecimalLength(n // 16)</pre>	435. Uniqueness of LRU Cache [Medium] Class Problem Given a list of operations, return <code>true</code> if the LRU cache is unique. Key Insights Use a hash map to store the frequency of operations. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(n)$	<pre>def isUniqueOperations(operations: List[List[str]]) -> bool: n = len(operations) for i in range(n): for j in range(i + 1, n): if operations[i] == operations[j]: return False return True</pre>	436. Find the Duplicate Number [Medium] Class Problem Given an array of numbers, return the duplicate number. Key Insights Use a Floyd's cycle-finding algorithm to find the duplicate number. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(1)$	<pre>def findDuplicate(nums: List[int]) -> int: n = len(nums) slow = fast = nums[0] while True: slow = nums[slow] fast = nums[nums[fast]] if slow == fast: break slow = nums[0] while slow != fast: slow = nums[slow] fast = nums[fast] return slow</pre>					
437. Path Sum III [Medium] Class Problem Given a binary tree, return the number of paths that sum to the target. Key Insights Use a recursive function to count the number of paths. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(n)$	<pre>def pathSum(root: Optional[TreeNode], targetSum: int) -> int: def dfs(node, currentSum): if not node: return 0 currentSum += node.val return dfs(node.left, currentSum) + dfs(node.right, currentSum) + (1 if currentSum == targetSum else 0) return dfs(root, 0)</pre>	438. Find All Paths from Root to Leaf [Medium] Class Problem Given a binary tree, return all paths from the root to the leaf. Key Insights Use a recursive function to find all paths. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(n)$	<pre>def binaryTreePaths(root: Optional[TreeNode]) -> List[str]: def dfs(node, path): if not node: return path.append(node.val) if not node.left and not node.right: result.append(' '.join(map(str, path))) dfs(node.left, path) dfs(node.right, path) path.pop() result = [] dfs(root, []) return result</pre>	439. Ternary Search Tree [Medium] Class Problem Given a list of numbers, return the ternary search tree. Key Insights Use a recursive function to build the ternary search tree. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(n)$	<pre>def ternarySearchTree(nums: List[int]) -> Optional[TreeNode]: def dfs(i): if i == len(nums): return None node = TreeNode(nums[i]) node.left = dfs(i + 1) node.middle = dfs(i + 1) node.right = dfs(i + 1) return node return dfs(0)</pre>	440. Koko Eating Bananas [Medium] Class Problem Given an array of numbers, return the minimum number of hours to eat all the bananas. Key Insights Use a binary search algorithm to find the minimum number of hours. Space & Time Time Complexity: $O(n \log \text{max_value})$ Space Complexity: $O(1)$	<pre>def minHoursToEatBananas(piles: List[int], h: int) -> bool: def canEat(piles, h, k): for pile in piles: hours = (pile + k - 1) // k if hours > h: return False return True left = 1 right = max(piles) while left < right: mid = (left + right) // 2 if canEat(piles, mid, k): right = mid else: left = mid + 1 return left</pre>	441. Arranging Coins [Medium] Class Problem Given a number <code>n</code>, return the number of complete rows that can be formed. Key Insights Use a binary search algorithm to find the number of complete rows. Space & Time Time Complexity: $O(\log n)$ Space Complexity: $O(1)$	<pre>def arrangeCoins(n: int) -> int: left = 0 right = n while left < right: mid = (left + right) // 2 if (mid * (mid + 1)) // 2 <= n: left = mid + 1 else: right = mid return left - 1</pre>	442. Find All Duplicates in an Array [Medium] Class Problem Given an array of numbers, return all the duplicate numbers. Key Insights Use a hash map to find the duplicate numbers. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(n)$	<pre>def findDuplicates(nums: List[int]) -> List[int]: n = len(nums) dp = [0] * n for num in nums: dp[num] += 1 result = [] for i in range(n): if dp[i] > 1: result.append(i + 1) return result</pre>	443. String Compression [Medium] Class Problem Given a string <code>s</code>, return the compressed string. Key Insights Use a recursive function to compress the string. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(n)$	<pre>def compressString(s: str) -> str: def dfs(i): if i == len(s): return '' count = 1 if s[i] == s[i + 1]: count += 1 return s[i] + dfs(i + 1) else: return s[i] + dfs(i + 1) return dfs(0)</pre>	444. Count of Range Numbers [Medium] Class Problem Given a range <code>[left, right]</code>, return the count of range numbers. Key Insights Use a recursive function to count the range numbers. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(1)$	<pre>def countRangeNumbers(left: int, right: int) -> int: def dfs(i, j): if i > j: return 0 if i == j: return 1 return dfs(i, i + 1) + dfs(i + 1, j) return dfs(left, right)</pre>					
445. Assign Cookies [Medium] Class Problem Given two arrays of numbers, return the maximum number of children that can be satisfied. Key Insights Use a greedy algorithm to maximize the number of children. Space & Time Time Complexity: $O(n \log n)$ Space Complexity: $O(1)$	<pre>def findContentChildren(g: List[int], s: List[int]) -> int: g.sort() s.sort() i = 0 j = 0 while i < len(g) and j < len(s): if s[j] < g[i]: j += 1 else: i += 1 return i</pre>	446. Arithmetic Slices II - Subsequence [Hard] Class Problem Given an array of numbers, return the number of arithmetic subsequences. Key Insights Use a dynamic programming approach to find the number of arithmetic subsequences. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(n^2)$	<pre>def numberOfArithmeticSlices(nums: List[int]) -> int: n = len(nums) dp = [[0] * n for _ in range(n)] for i in range(n): for j in range(i + 1, n): for k in range(j + 1, n): if nums[i] + nums[k] == 2 * nums[j]: dp[i][j] += dp[i][k] + 1 dp[j][k] += 1 return sum(dp[i][j] for i in range(n) for j in range(i + 1, n))</pre>	447. Number of Boats to Save People [Medium] Class Problem Given an array of numbers, return the minimum number of boats to save all the people. Key Insights Use a greedy algorithm to minimize the number of boats. Space & Time Time Complexity: $O(n \log n)$ Space Complexity: $O(1)$	<pre>def minBoatsToSavePeople(people: List[int], limit: int) -> int: people.sort() i = 0 j = len(people) - 1 boats = 0 while i < j: if people[i] + people[j] > limit: boats += 1 j -= 1 else: i += 1 boats += 1 return boats</pre>	448. Find All Possible Recipes from Given Supplies [Medium] Class Problem Given a list of ingredients and recipes, return all possible recipes. Key Insights Use a recursive function to find all possible recipes. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(n)$	<pre>def findAllRecipes(ingredients: List[str], recipes: List[List[str]], supplies: List[str]) -> List[str]: graph = defaultdict(list) for recipe in recipes: for ingredient in recipe[1:]: graph[ingredient].append(recipe[0]) result = [] def dfs(ingredient): if ingredient in supplies: return True for recipe in graph[ingredient]: if dfs(recipe): result.append(recipe) for ingredient in ingredients: dfs(ingredient) return result</pre>	449. Serialize and Deserialize Binary Tree [Hard] Class Problem Serialize and deserialize a binary tree. Key Insights Use a recursive function to serialize and deserialize the tree. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(n)$	<pre>def serialize(root: Optional[TreeNode]) -> str: if not root: return '#' return str(root.val) + ',' + serialize(root.left) + ',' + serialize(root.right) def deserialize(data: str) -> Optional[TreeNode]: if not data: return None data = data.split(',') root = TreeNode(int(data[0])) root.left = deserialize(data[2:]) root.right = deserialize(data[4:]) return root</pre>	450. Delete Node in a Binary Search Tree [Medium] Class Problem Delete a node from a binary search tree. Key Insights Use a recursive function to delete the node. Space & Time Time Complexity: $O(h)$ Space Complexity: $O(h)$	<pre>def deleteNode(root: Optional[TreeNode], key: int) -> Optional[TreeNode]: if not root: return None if key < root.val: root.left = deleteNode(root.left, key) elif key > root.val: root.right = deleteNode(root.right, key) else: if not root.left: return root.right if not root.right: return root.left minNode = root.right while minNode.left: minNode = minNode.left root.val = minNode.val root.right = deleteNode(root.right, minNode.val) return root</pre>	451. Sort Characters By Frequency [Medium] Class Problem Sort the characters of a string by their frequency. Key Insights Use a hash map to count the frequency of characters. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(1)$	<pre>def sortCharactersByFrequency(s: str) -> str: count = {} for c in s: count[c] = count.get(c, 0) + 1 sorted_chars = sorted(count.items(), key=lambda x: x[1], reverse=True) result = '' for char, freq in sorted_chars: result += char * freq return result</pre>	452. Minimum Number of Arrows to Burst Balloons [Medium] Class Problem Given an array of numbers, return the minimum number of arrows to burst all the balloons. Key Insights Use a greedy algorithm to minimize the number of arrows. Space & Time Time Complexity: $O(n \log n)$ Space Complexity: $O(1)$	<pre>def findMinArrowShots(points: List[List[int]]) -> int: points.sort() arrows = 1 end = points[0][1] for i in range(1, len(points)): if points[i][0] > end: arrows += 1 end = points[i][1] return arrows</pre>	453. Minimum Moves to Equal Array Elements [Medium] Class Problem Given an array of numbers, return the minimum number of moves to make all elements equal. Key Insights Use a greedy algorithm to minimize the number of moves. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(1)$	<pre>def minMoves(nums: List[int]) -> int: n = len(nums) sum_nums = sum(nums) avg = sum_nums // n moves = 0 for num in nums: moves += abs(num - avg) return moves</pre>	454. 4Sum [Medium] Class Problem Given an array of numbers, return all the quadruplets that sum to the target. Key Insights Use a recursive function to find all quadruplets. Space & Time Time Complexity: $O(n^4)$ Space Complexity: $O(1)$	<pre>def fourSum(nums: List[int], target: int) -> List[List[int]]: n = len(nums) for i in range(n): for j in range(i + 1, n): for k in range(j + 1, n): for l in range(k + 1, n): if nums[i] + nums[j] + nums[k] + nums[l] == target: result.append([nums[i], nums[j], nums[k], nums[l]]) return result</pre>	455. Assign Cookies [Medium] Class Problem Given two arrays of numbers, return the maximum number of children that can be satisfied. Key Insights Use a greedy algorithm to maximize the number of children. Space & Time Time Complexity: $O(n log$

#112 Path Sum LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary tree and a target sum, return true if there is a path from the root to a leaf node such that the sum of the values along the path equals the target sum. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#126 Invert Binary Tree LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary tree, invert it (swap left and right children of every node). Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#230 Kth Smallest Element in a BST LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary search tree, return the kth smallest element. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#235 Binary Search Tree Lowest Common Ancestor LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary search tree and two nodes, return their lowest common ancestor. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#297 Serialize and Deserialize Binary Tree LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Implement a method to serialize a binary tree and a method to deserialize it back into a binary tree. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#437 Path Sum II LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary tree and a target sum, return all root-to-leaf paths where the sum of the values equals the target sum. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#513 Find Bottom Element in a Binary Tree LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary tree, return the value of the bottom-most node. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#543 Diameter of Binary Tree LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary tree, return the diameter of the tree (the length of the longest path between any two nodes). Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#572 Subtree of Another Tree LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given two binary trees, return true if the first tree is a subtree of the second tree. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#606 Construct Binary Tree from Preorder and Inorder Traversal LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given the preorder and inorder traversal of a binary tree, construct the tree. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#607 Construct Binary Tree from Preorder and Postorder Traversal LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given the preorder and postorder traversal of a binary tree, construct the tree. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#637 Average of Levels in Binary Tree LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary tree, return the average value of the nodes at each level. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#653 Two Sum IV - Input is a BST LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary search tree and a target sum, return true if there are two nodes in the tree whose values sum to the target. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#654 Maximum Binary Tree LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given an array of integers, construct a maximum binary tree from it. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#655 Print Binary Tree LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary tree, print its level-order traversal. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#659 Split Binary Tree with Largest Product LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a binary tree, split it into two subtrees such that the product of the sums of the nodes in each subtree is maximized. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre>	#661 Image Smoother LeetCode - Simplified - WACC - LeetCode Time: O(N) - O(1) Space: O(N) - O(1) Problem: Given a 2D grid of integers, return a smoothed version of the grid. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --></pre>
---	--	---	---	--	---	--	---	---	---	---	---	--	---	--	--	--

[illegible]

Tree & BST

#238 Binary Tree Longest Consecutive Sequence

LeetCode - Simplified - WACC - LeetCode

Tree - 100%

Like Problem 98

Time Complexity: O(N), where N is the number of nodes in the tree, since each node is visited once. Space Complexity: O(N), where N is the height of the tree in the worst case of a skewed tree. That's it.

Problem

Given the root of a binary tree, return the length of the longest consecutive sequence path. A path where the values increase by one, or decrease by one, is considered a valid path. Note that the path can start at any node in the tree, and you cannot go from null to its parent to the next node.

Example 1:

Input: root = [3,1,4,null,2,null,6,4,null,0]

Output: 4

Explanation: The longest consecutive sequence path is 1 -> 2 -> 3 -> 4, with a length of 4. Note that the path 4 -> 2 -> 0 is not valid, because the values are decreasing.

Example 2:

Input: root = [1,0,1,null,1,null,2,null,1,null,1,0,0]

Output: 2

Explanation: The longest consecutive sequence path is 1 -> 2, with a length of 2.

Constraints:

• The number of nodes in the tree is in the range [1, 3 * 10⁴].

• -100 <= Node.val <= 100

Tree & BST

Community Solutions (Python)

WACC

def longestConsecutive(self, root: Optional[TreeNode]) -> int: if not root: return 0 def dfs(node: Optional[TreeNode], prev: int, length: int) -> int: if not node: return length if node.val == prev + 1: return dfs(node.left, node.val, length + 1) elif node.val == prev - 1: return dfs(node.right, node.val, length + 1) else: return 1 + max(dfs(node.left, node.val, 1), dfs(node.right, node.val, 1)) return 1 + max(dfs(root, root.val, 1), dfs(root, root.val, 1))

LeafNodes

def longestConsecutive(self, root: Optional[TreeNode]) -> int: if not root: return 0 def dfs(node: Optional[TreeNode], prev: int, length: int) -> int: if not node: return length if node.val == prev + 1: return dfs(node.left, node.val, length + 1) elif node.val == prev - 1: return dfs(node.right, node.val, length + 1) else: return 1 + max(dfs(node.left, node.val, 1), dfs(node.right, node.val, 1)) return 1 + max(dfs(root, root.val, 1), dfs(root, root.val, 1))

Tree & BST

#314 Binary Tree Vertical Order Traversal

LeetCode - Simplified - WACC - LeetCode

Tree - 100%

Like Problem 98

Time Complexity: O(N log N), where N is the number of nodes in the tree, since each node is visited once. Space Complexity: O(N), where N is the height of the tree in the worst case of a skewed tree. That's it.

Problem

Given the root of a binary tree, return the vertical order traversal of its nodes' values. (i.e., from top to bottom, column by column). If two nodes are in the same row and column, the order should be from left to right.

Example 1:

Input: root = [3,9,20,null,null,15,7]

Output: [[3], [9,15], [20], [7]]

Example 2:

Input: root = [3,9,8,4,0,1,7]

Output: [[3], [9,4], [15,8], [20], [7]]

Constraints:

• The number of nodes in the tree is in the range [1, 1000].

• -1000 <= Node.val <= 1000

Tree & BST

Community Solutions (Python)

WACC

def verticalTraversal(self, root: Optional[TreeNode]) -> List[List[int]]: if not root: return [] def dfs(node: Optional[TreeNode], col: int, row: int, res: List[List[int]]) -> None: if not node: return res[col].append((row, node.val)) dfs(node.left, col - 1, row + 1, res) dfs(node.right, col + 1, row + 1, res) return res def sort(res: List[List[int]]) -> List[List[int]]: return sorted(res, key=lambda x: (x[0], x[1])) return [val for _, val in sort(res)]

LeafNodes

def verticalTraversal(self, root: Optional[TreeNode]) -> List[List[int]]: if not root: return [] def dfs(node: Optional[TreeNode], col: int, row: int, res: List[List[int]]) -> None: if not node: return res[col].append((row, node.val)) dfs(node.left, col - 1, row + 1, res) dfs(node.right, col + 1, row + 1, res) return res def sort(res: List[List[int]]) -> List[List[int]]: return sorted(res, key=lambda x: (x[0], x[1])) return [val for _, val in sort(res)]

Tree & BST

#437 Path Sum III

LeetCode - Simplified - WACC - LeetCode

Tree - 100%

Like Problem 98

Time Complexity: O(N), where N is the number of nodes in the tree, since each node is visited once. Space Complexity: O(N), where N is the height of the tree in the worst case of a skewed tree. That's it.

Problem

Given the root of a binary tree, return the number of paths that sum to the target. The paths can start at any node in the tree, and you cannot go from null to its parent to the next node.

Example 1:

Input: root = [10,5,-10,4,-10,9,-10,1]

Output: 3

Example 2:

Input: root = [10,5,-10,4,-10,9,-10,1]

Output: 3

Constraints:

• The number of nodes in the tree is in the range [1, 1000].

• -1000 <= Node.val <= 1000

Tree & BST

Community Solutions (Python)

WACC

def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int: if not root: return 0 def dfs(node: Optional[TreeNode], targetSum: int, res: int) -> int: if not node: return res if node.val == targetSum: res += 1 res = dfs(node.left, targetSum, res) res = dfs(node.right, targetSum, res) return res return dfs(root, targetSum, 0)

LeafNodes

def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int: if not root: return 0 def dfs(node: Optional[TreeNode], targetSum: int, res: int) -> int: if not node: return res if node.val == targetSum: res += 1 res = dfs(node.left, targetSum, res) res = dfs(node.right, targetSum, res) return res return dfs(root, targetSum, 0)

Tree & BST

#336 Find Leaves of Binary Tree

LeetCode - Simplified - WACC - LeetCode

Tree - 100%

Like Problem 98

Time Complexity: O(N), where N is the number of nodes in the tree, since each node is visited once. Space Complexity: O(N), where N is the height of the tree in the worst case of a skewed tree. That's it.

Problem

Given the root of a binary tree, collect a tree's nodes as if you were doing a leaf collection. (i.e., from the leaf nodes, up to the root node, and so on, until there are no more leaf nodes.) Return a list of lists of the nodes of the tree at each stage of the collection process. (i.e., in the order in which they were removed from the tree.)

Example 1:

Input: root = [1,2,3,4,5]

Output: [[4,5], [3], [2], [1]]

Example 2:

Input: root = [1,2,3,4,5,6]

Output: [[6], [5,4], [3], [2], [1]]

Constraints:

• The number of nodes in the tree is in the range [1, 1000].

• -1000 <= Node.val <= 1000

Tree & BST

Community Solutions (Python)

WACC

def findLeaves(self, root: Optional[TreeNode]) -> List[List[int]]: if not root: return [] def dfs(node: Optional[TreeNode], level: int, res: List[List[int]]) -> None: if not node: return if not node.left and not node.right: res[level].append(node.val) else: dfs(node.left, level + 1, res) dfs(node.right, level + 1, res) return res return dfs(root, 0, [])

LeafNodes

def findLeaves(self, root: Optional[TreeNode]) -> List[List[int]]: if not root: return [] def dfs(node: Optional[TreeNode], level: int, res: List[List[int]]) -> None: if not node: return if not node.left and not node.right: res[level].append(node.val) else: dfs(node.left, level + 1, res) dfs(node.right, level + 1, res) return res return dfs(root, 0, [])

Tree & BST

#437 Path Sum III

LeetCode - Simplified - WACC - LeetCode

Tree - 100%

Like Problem 98

Time Complexity: O(N), where N is the number of nodes in the tree, since each node is visited once. Space Complexity: O(N), where N is the height of the tree in the worst case of a skewed tree. That's it.

Problem

Given the root of a binary tree, return the number of paths that sum to the target. The paths can start at any node in the tree, and you cannot go from null to its parent to the next node.

Example 1:

Input: root = [10,5,-10,4,-10,9,-10,1]

Output: 3

Example 2:

Input: root = [10,5,-10,4,-10,9,-10,1]

Output: 3

Constraints:

• The number of nodes in the tree is in the range [1, 1000].

• -1000 <= Node.val <= 1000

Tree & BST

Community Solutions (Python)

WACC

def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int: if not root: return 0 def dfs(node: Optional[TreeNode], targetSum: int, res: int) -> int: if not node: return res if node.val == targetSum: res += 1 res = dfs(node.left, targetSum, res) res = dfs(node.right, targetSum, res) return res return dfs(root, targetSum, 0)

LeafNodes

def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int: if not root: return 0 def dfs(node: Optional[TreeNode], targetSum: int, res: int) -> int: if not node: return res if node.val == targetSum: res += 1 res = dfs(node.left, targetSum, res) res = dfs(node.right, targetSum, res) return res return dfs(root, targetSum, 0)

Tree & BST

#437 Path Sum III

LeetCode - Simplified - WACC - LeetCode

Tree - 100%

Like Problem 98

Time Complexity: O(N), where N is the number of nodes in the tree, since each node is visited once. Space Complexity: O(N), where N is the height of the tree in the worst case of a skewed tree. That's it.

Problem

Given the root of a binary tree, return the number of paths that sum to the target. The paths can start at any node in the tree, and you cannot go from null to its parent to the next node.

Example 1:

Input: root = [10,5,-10,4,-10,9,-10,1]

Output: 3

Example 2:

Input: root = [10,5,-10,4,-10,9,-10,1]

Output: 3

Constraints:

• The number of nodes in the tree is in the range [1, 1000].

• -1000 <= Node.val <= 1000

Tree & BST

Community Solutions (Python)

WACC

def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int: if not root: return 0 def dfs(node: Optional[TreeNode], targetSum: int, res: int) -> int: if not node: return res if node.val == targetSum: res += 1 res = dfs(node.left, targetSum, res) res = dfs(node.right, targetSum, res) return res return dfs(root, targetSum, 0)

LeafNodes

def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int: if not root: return 0 def dfs(node: Optional[TreeNode], targetSum: int, res: int) -> int: if not node: return res if node.val == targetSum: res += 1 res = dfs(node.left, targetSum, res) res = dfs(node.right, targetSum, res) return res return dfs(root, targetSum, 0)

Tree & BST

#545 Boundary of Binary Tree

LeetCode - Simplified - WACC - LeetCode

Tree - 100%

Like Problem 98

Time Complexity: O(N), where N is the number of nodes in the tree, since each node is visited once. Space Complexity: O(N), where N is the height of the tree in the worst case of a skewed tree. That's it.

Problem

Given the root of a binary tree, return the values of its boundary nodes. The boundary nodes are the nodes that are on the left boundary, the nodes that are on the right boundary, and the leaf nodes. The leaf nodes are the nodes that have no children.

Example 1:

Input: root = [1,2,3,4,5,6,7]

Output: [1,2,4,7,5,6,3]

Example 2:

Input: root = [1,2,3,4,5,6,7]

Output: [1,2,4,7,5,6,3]

Constraints:

• The number of nodes in the tree is in the range [1, 1000].

• -1000 <= Node.val <= 1000

Tree & BST

Community Solutions (Python)

WACC

def boundary(self, root: Optional[TreeNode]) -> List[int]: if not root: return [] def dfs(node: Optional[TreeNode], level: int, res: List[int]) -> None: if not node: return if not node.left and not node.right: res.append(node.val) else: dfs(node.left, level + 1, res) dfs(node.right, level + 1, res) return res return dfs(root, 0, [])

LeafNodes

def boundary(self, root: Optional[TreeNode]) -> List[int]: if not root: return [] def dfs(node: Optional[TreeNode], level: int, res: List[int]) -> None: if not node: return if not node.left and not node.right: res.append(node.val) else: dfs(node.left, level + 1, res) dfs(node.right, level + 1, res) return res return dfs(root, 0, [])

Tree & BST

#545 Boundary of Binary Tree

LeetCode - Simplified - WACC - LeetCode

Tree - 100%

Like Problem 98

Time Complexity: O(N), where N is the number of nodes in the tree, since each node is visited once. Space Complexity: O(N), where N is the height of the tree in the worst case of a skewed tree. That's it.

Problem

Given the root of a binary tree, return the values of its boundary nodes. The boundary nodes are the nodes that are on the left boundary, the nodes that are on the right boundary, and the leaf nodes. The leaf nodes are the nodes that have no children.

Example 1:

Input: root = [1,2,3,4,5,6,7]

Output: [1,2,4,7,5,6,3]

Example 2:

Input: root = [1,2,3,4,5,6,7]

Output: [1,2,4,7,5,6,3]

Constraints:

• The number of nodes in the tree is in the range [1, 1000].

• -1000 <= Node.val <= 1000

Tree & BST

Community Solutions (Python)

WACC

def boundary(self, root: Optional[TreeNode]) -> List[int]: if not root: return [] def dfs(node: Optional[TreeNode], level: int, res: List[int]) -> None: if not node: return if not node.left and not node.right: res.append(node.val) else: dfs(node.left, level + 1, res) dfs(node.right, level + 1, res) return res return dfs(root, 0, [])

LeafNodes

def boundary(self, root: Optional[TreeNode]) -> List[int]: if not root: return [] def dfs(node: Optional[TreeNode], level: int, res: List[int]) -> None: if not node: return if not node.left and not node.right: res.append(node.val) else: dfs(node.left, level + 1, res) dfs(node.right, level + 1, res) return res return dfs(root, 0, [])

Tree & BST

#545 Boundary of Binary Tree

LeetCode - Simplified - WACC - LeetCode

Tree - 100%

Like Problem 98

Time Complexity: O(N), where N is the number of nodes in the tree, since each node is visited once. Space Complexity: O(N), where N is the height of the tree in the worst case of a skewed tree. That's it.

Problem

Given the root of a binary tree, return the values of its boundary nodes. The boundary nodes are the nodes that are on the left boundary, the nodes that are on the right boundary, and the leaf nodes. The leaf nodes are the nodes that have no children.

Example 1:

Input: root = [1,2,3,4,5,6,7]

Output: [1,2,4,7,5,6,3]

Example 2:

Input: root = [1,2,3,4,5,6,7]

Output: [1,2,4,7,5,6,3]

Constraints:

• The number of nodes in the tree is in the range [1, 1000].

• -1000 <= Node.val <= 1000

Tree & BST

Community Solutions (Python)

WACC

def boundary(self, root: Optional[TreeNode]) -> List[int]: if not root: return [] def dfs(node: Optional[TreeNode], level: int, res: List[int]) -> None: if not node: return if not node.left and not node.right: res.append(node.val) else: dfs(node.left, level + 1, res) dfs(node.right, level + 1, res) return res return dfs(root, 0, [])

LeafNodes

def boundary(self, root: Optional[TreeNode]) -> List[int]: if not root: return [] def dfs(node: Optional[TreeNode], level: int, res: List[int]) -> None: if not node: return if not node.left and not node.right: res.append(node.val) else: dfs(node.left, level + 1, res) dfs(node.right, level + 1, res) return res return dfs(root, 0, [])

[illegible]

